

CSC-3TC33

Algorithmique et fondements de l'informatique

Manipulations de chaînes de caractères - partie a

Bertrand Meyer

Automne 2024

Alphabet (révisions)

Définition

Un **alphabet** est un ensemble fini de symboles.

Exemple

$\Sigma = \{a, b\}$

Σ = ensemble des symboles de la table ASCII.



Les mots (révisions)

Définition

Un **mot** sur l'alphabet Σ est une suite finie de symbole de Σ .

L'ensemble des mots se note Σ^* ,
l'ensemble des mots non vides se note Σ^+ , .

Exemple

Le mot **abbab** est un mot sur l'alphabet $\Sigma = \{a, b\}$.

Définition

La longueur d'un **mot** est son nombre de lettre.

Exemple

Le mot **abbab** est de longueur 5.

Le mot vide se note ε : il est de longueur nulle.



Définition

Lorsqu'un mot est de la forme $w = uv$ avec $u \in \Sigma^*$ et $v \in \Sigma^*$, on dit que u est un **préfixe** de w et que v est un **suffixe** de w .

Définition

Lorsqu'un mot est de la forme $w = uv$ avec $u \in \Sigma^*$ et $v \in \Sigma^*$, on dit que u est un **préfixe** de w et que v est un **suffixe** de w .

Définition

Lorsqu'un mot est de la forme $w = u_1vu_2$ avec $u_1, v, u_2 \in \Sigma^*$, on dit que v est un **facteur** de w .

Remarque

En informatique, on parle aussi de **sous-chaîne de caractère**.

Définition

Lorsqu'un mot est de la forme $w = uv$ avec $u \in \Sigma^*$ et $v \in \Sigma^*$, on dit que u est un **préfixe** de w et que v est un **suffixe** de w .

Définition

Lorsqu'un mot est de la forme $w = u_1vu_2$ avec $u_1, v, u_2 \in \Sigma^*$, on dit que v est un **facteur** de w .

Remarque

En informatique, on parle aussi de **sous-chaîne de caractère**.

Définition

Lorsqu'un mot est de la forme $w = x_1x_2 \dots x_n$ avec $x_1, x_2, \dots, x_n \in \Sigma$, on dit que $x_{i_1}x_{i_2} \dots x_{i_t}$ est un **sous-mot** de w si $i_1 < i_2 < \dots < i_t$.

Reconnaître un motif

Le problème de la sous-chaîne

Définition

Étant donné deux mots s et p sur un alphabet Σ , le problème de la **recherche de sous-chaîne** consiste à déterminer si le motif p est un *facteur* de s et, éventuellement, localiser p dans s .

Le problème de la sous-chaîne

Définition

Étant donné deux mots s et p sur un alphabet Σ , le problème de la **recherche de sous-chaîne** consiste à déterminer si le motif p est un *facteur* de s et, éventuellement, localiser p dans s .

Exemple

Dans la chaîne

$$s = \text{babaababababcb},$$

le motif $p = \text{abababc}$ apparaît en position 6.

Le problème de la sous-chaîne

Définition

Étant donné deux mots s et p sur un alphabet Σ , le problème de la **recherche de sous-chaîne** consiste à déterminer si le motif p est un *facteur* de s et, éventuellement, localiser p dans s .

Exemple

Dans la chaîne

$s = \text{babaababababcab},$

le motif $p = \text{abababc}$ apparaît en position 6.

Recherche par force brute

Principe

On fait glisser le motif p le long de la chaîne s :

On compare caractère par caractère.

En cas d'échec, on décale d'un pas de 1.

```
def bruteforce(s, p):  
    n = len(s)  
    m = len(p)  
    for i in range(n-m+1):  
        j = 0  
        while s[i+j] == p[j] and j < m:  
            j += 1  
        if j == m:  
            return(i)  
    return("No match")
```



Recherche par force brute

Principe

On fait glisser le motif p le long de la chaîne s :

On compare caractère par caractère.

En cas d'échec, on décale d'un pas de 1.

```
def bruteforce(s, p):  
    n = len(s)  
    m = len(p)  
    for i in range(n-m+1):  
        j = 0  
        while s[i+j] == p[j] and j < m:  
            j += 1  
        if j == m:  
            return(i)  
    return("No match")
```



Complexité : $O(|s| \cdot |p|)$.

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Non concordance : on va décaler p de 1

a						
---	--	--	--	--	--	--

$W = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Concordance : p ne bouge pas

a						
---	--	--	--	--	--	--

$w = \epsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcbcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

↕ Concordance : p ne bouge pas

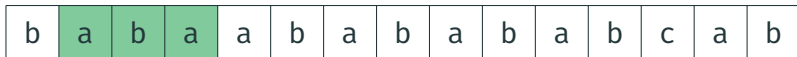
a	b					
---	---	--	--	--	--	--



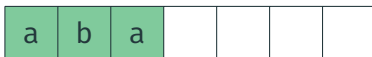
accord déjà constaté auparavant sur 1 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcb}$



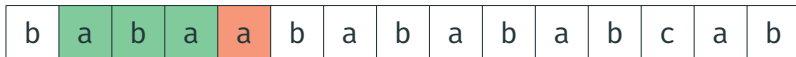
↕ Concordance : p ne bouge pas



accord déjà constaté auparavant sur 2 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcb}$



Non concordance : on va décaler p de 1



w

accord déjà constaté auparavant sur 3 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Non concordance : on va décaler p de 1

a						
---	--	--	--	--	--	--

$w = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \mathbf{abababc}$ dans la chaîne
 $s = \mathbf{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Concordance : p ne bouge pas

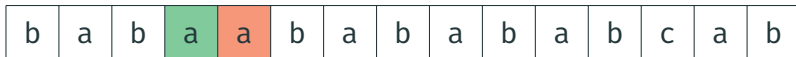
a						
---	--	--	--	--	--	--

$w = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcbcab}$



Non concordance : on va décaler p de 1



w

accord déjà constaté auparavant sur 1 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcb}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Concordance : p ne bouge pas

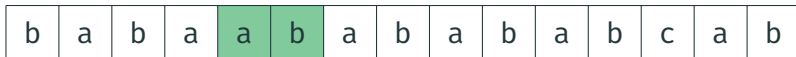
a						
---	--	--	--	--	--	--

$w = \epsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcbcab}$



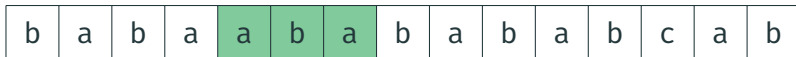
↕ Concordance : p ne bouge pas



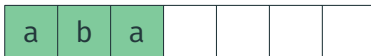
accord déjà constaté auparavant sur 1 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcb}$



Concordance : p ne bouge pas

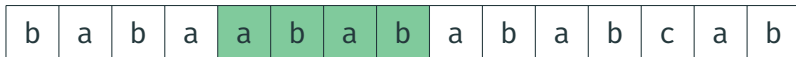


w

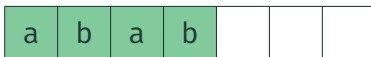
accord déjà constaté auparavant sur 2 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



Concordance : p ne bouge pas

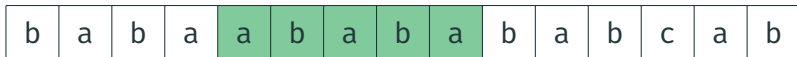


w

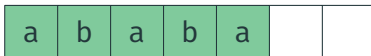
accord déjà constaté auparavant sur 3 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



Concordance : p ne bouge pas

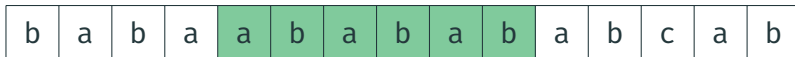


w

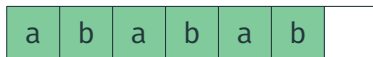
accord déjà constaté auparavant sur 4 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



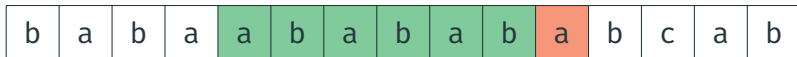
↕ Concordance : p ne bouge pas



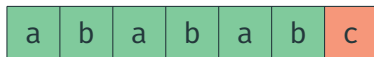
accord déjà constaté auparavant sur 5 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



↕ Non concordance : on va décaler p de 1



accord déjà constaté auparavant sur 6 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Non concordance : on va décaler p de 1

a	b	a	b	a	b	c
---	---	---	---	---	---	---

$w = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

↕ Concordance

a						
---	--	--	--	--	--	--

$W = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

↕ Concordance

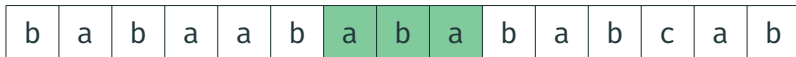
a	b					
---	---	--	--	--	--	--

↔
 w

accord déjà constaté auparavant sur 1 lettre

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



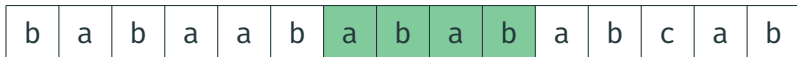
↕ Concordance



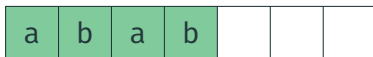
accord déjà constaté auparavant sur 2 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



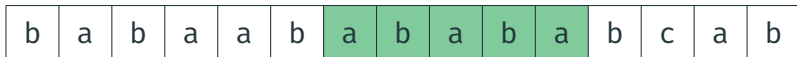
↕ Concordance



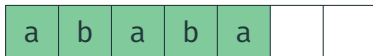
accord déjà constaté auparavant sur 3 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



↕ Concordance

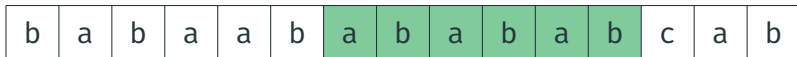


w

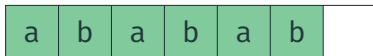
accord déjà constaté auparavant sur 4 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



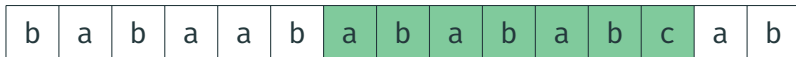
↕ Concordance



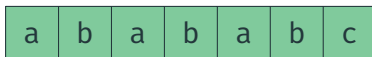
accord déjà constaté auparavant sur 5 lettres

Illustration : recherche par force brute

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



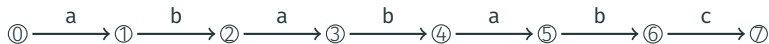
↕ Concordance



accord déjà constaté auparavant sur 6 lettres

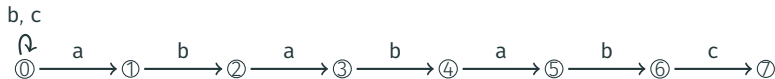
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



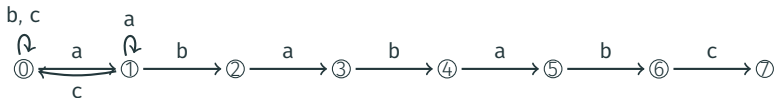
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



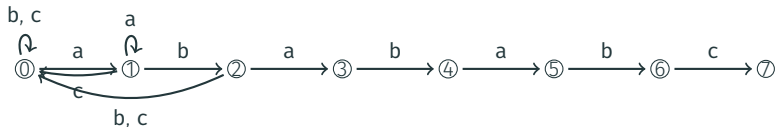
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



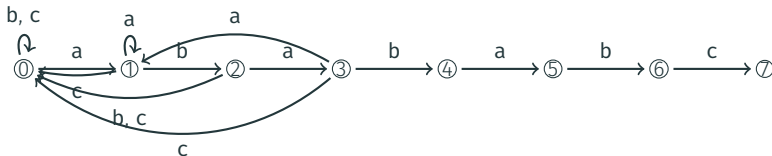
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



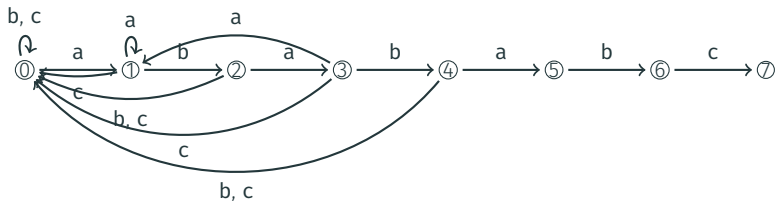
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



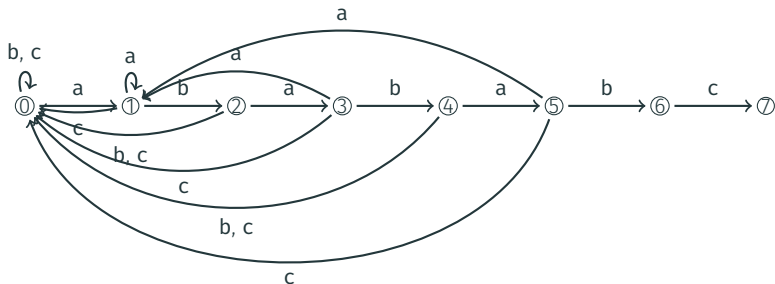
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



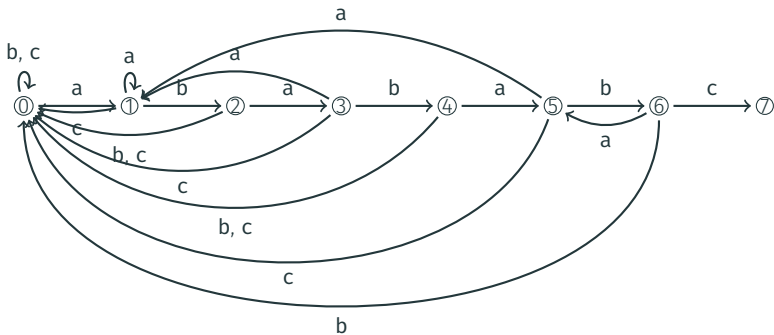
Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



Idée d'amélioration : vision graphique

À chaque décalage du motif, on souhaite profiter du fait que l'on a reconnu un préfixe de p pour ne pas recommencer à zéro les comparaisons.



Définition

Un **bord** d'un mot $w \in \Sigma^+$ tout facteur propre de w qui est à la fois préfixe et suffixe de w .

Définition

Un **bord** d'un mot $w \in \Sigma^+$ tout facteur propre de w qui est à la fois préfixe et suffixe de w .

Exemple

Le mot $w = \text{acaacababaacaaca}$ a pour ensemble de bords

Définition

Un **bord** d'un mot $w \in \Sigma^+$ tout facteur propre de w qui est à la fois préfixe et suffixe de w .

Exemple

Le mot $w = \text{acaacababaacaaca}$ a pour ensemble de bords $\{\varepsilon, \text{a}, \text{aca}, \text{acaaca}\}$;

Définition

Un **bord** d'un mot $w \in \Sigma^+$ tout facteur propre de w qui est à la fois préfixe et suffixe de w .

Exemple

Le mot $w = \text{acaacababaacaaca}$ a pour ensemble de bords

$\{\varepsilon, \mathbf{a}, \mathbf{aca}, \mathbf{acaaca}\};$

le mot $w = \mathbf{ababab}$ a pour ensemble de bords

Définition

Un **bord** d'un mot $w \in \Sigma^+$ tout facteur propre de w qui est à la fois préfixe et suffixe de w .

Exemple

Le mot $w = \text{acaacababaacaaca}$ a pour ensemble de bords

$\{\varepsilon, \mathbf{a}, \mathbf{aca}, \mathbf{acaaca}\}$;

le mot $w = \mathbf{ababab}$ a pour ensemble de bords $\{\varepsilon, \mathbf{ab}, \mathbf{abab}\}$.

Bord le plus long d'un mot

On note $\text{bord}(w)$ le **plus long bord** de w .

Lemme

L'ensemble des bords d'un mot $w \in \Sigma^+$ est de la forme

$$\{\text{bord}(w), \text{bord}(\text{bord}(w)), \dots, \text{bord}^k(w)\}$$

où k est le plus petit entier tel que $\text{bord}^k(w)$ est le mot vide.

Bord le plus long d'un mot

On note $\text{bord}(w)$ le **plus long bord** de w .

Lemme

L'ensemble des bords d'un mot $w \in \Sigma^+$ est de la forme

$$\{\text{bord}(w), \text{bord}(\text{bord}(w)), \dots, \text{bord}^k(w)\}$$

où k est le plus petit entier tel que $\text{bord}^k(w)$ est le mot vide.

Exemple

Le mot $w = \text{acaacababaacaaca}$ a pour ensemble de bords $\{\varepsilon, a, aca, acaaca\}$. On a

$$\begin{aligned}\text{bord}(\underline{\text{acaacababaacaaca}}) &= \text{acaaca} \\ \text{bord}(\underline{\text{acaaca}}) &= \text{aca} \\ \text{bord}(\underline{\text{aca}}) &= a \\ \text{bord}(a) &= \varepsilon\end{aligned}$$

Calcul du plus long bord d'un mot

Soit $w = x_0x_1 \cdots x_i \in \Sigma^*$ le préfixe de p de longueur $i + 1$. On note $s(i + 1)$ la **longueur du plus long bord** de w .

Calcul du plus long bord d'un mot

Soit $w = x_0x_1 \cdots x_i \in \Sigma^*$ le préfixe de p de longueur $i + 1$. On note $s(i + 1)$ la **longueur du plus long bord** de w .

Un bord b non vide de w est forcément de la forme $b = ux_i$ où $u \in \Sigma^*$ est un bord de $x_0x_1 \cdots x_{i-1}$.



Calcul du plus long bord d'un mot

Soit $w = x_0x_1 \cdots x_i \in \Sigma^*$ le préfixe de p de longueur $i + 1$. On note $s(i + 1)$ la **longueur du plus long bord** de w .

Un bord b non vide de w est forcément de la forme $b = ux_i$ où $u \in \Sigma^*$ est un bord de $x_0x_1 \cdots x_{i-1}$.



On a donc

$$s(i + 1) = \begin{cases} 1 + \underbrace{s \circ s \circ \cdots \circ s(i)}_{k \text{ fois}} & \text{où } k \text{ plus petit entier t.q. } x_{s^k(i)} = x_i \\ 0 & \text{sinon} \end{cases}$$

Par convention, on pose $s(0) = -1$.

Calcul du plus long bord d'un mot : exemple

i	0	1	2	3	4	5	6	7
w	ε	a	ab	<u>a</u> $\bar{b}$	<u>a</u> <u>b</u> $\bar{a}$ $\bar{b}$	<u>a</u> <u>b</u> <u>a</u> $\bar{b}$	<u>a</u> <u>b</u> <u>a</u> <u>b</u> $\bar{a}$ $\bar{b}$	abababc
bord(w)	$\perp$	ε	ε	a	ab	aba	abab	ε
$s(i)$	-1	0	0	1	2	3	4	0

Première amélioration : Morris Pratt

~1970



James Hiram Morris
(1941)

Principe

On fait glisser le motif p le long de la chaîne s :

- On compare p à s caractère par caractère.
- En cas d'échec, on se recale sur le plus grand bord.
Autrement dit, si w est le préfixe de p qui a été reconnu, on se décale de $|w| - s(w)$.



Vaughan Ronald Pratt
(1944)

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Non concordance : on va décaler p de $|w| - s(|w|) = 0 - (-1) = 1$.

a						
---	--	--	--	--	--	--

$w = \varepsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \mathbf{abababc}$ dans la chaîne
 $s = \mathbf{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Concordance : p ne bouge pas

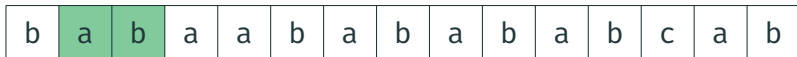
a						
---	--	--	--	--	--	--

$w = \epsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcbcab}$



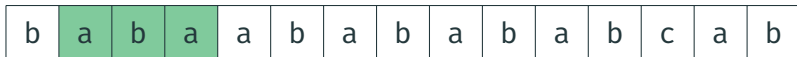
↕ Concordance : p ne bouge pas



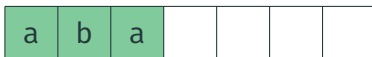
accord déjà constaté auparavant sur 1 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \mathbf{abababc}$ dans la chaîne
 $s = \mathbf{babaababababcab}$



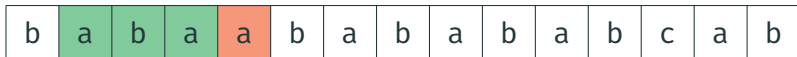
↕ Concordance : p ne bouge pas



accord déjà constaté auparavant sur 2 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \mathbf{abababc}$ dans la chaîne
 $s = \mathbf{babaababababcab}$



Non concordance : on va décaler p de $|w| - s(|w|) = 3 - (1) = 2$.



w

accord déjà constaté auparavant sur 3 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \mathbf{abababc}$ dans la chaîne
 $s = \mathbf{babaababababcbab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Non concordance : on va décaler p de $|w| - s(|w|) = 3 - (1) = 2$.

a	b	a	b			
---	---	---	---	--	--	--

a						
---	--	--	--	--	--	--

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcbcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

p a été décalé, on capitalise 1 comparaison antérieure

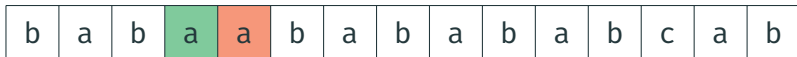
a						
---	--	--	--	--	--	--



accord déjà constaté auparavant sur 1 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



Non concordance : on va décaler p de $|w| - s(|w|) = 1 - 0 = 1$.



w

accord déjà constaté auparavant sur 1 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Concordance : p ne bouge pas

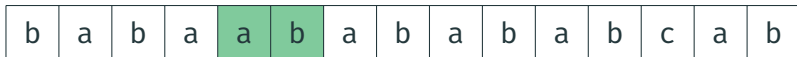
a						
---	--	--	--	--	--	--

$w = \epsilon$

accord déjà constaté auparavant sur 0 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



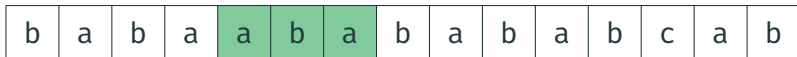
↕ Concordance : p ne bouge pas



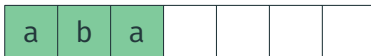
accord déjà constaté auparavant sur 1 lettre

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



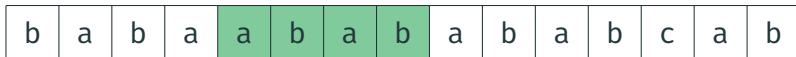
Concordance : p ne bouge pas



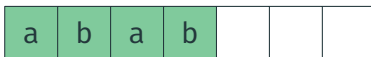
accord déjà constaté auparavant sur 2 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



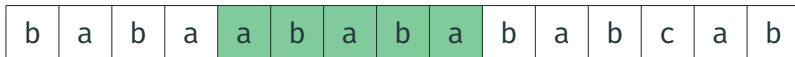
Concordance : p ne bouge pas



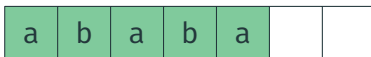
accord déjà constaté auparavant sur 3 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



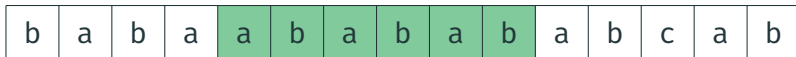
↕ Concordance : p ne bouge pas



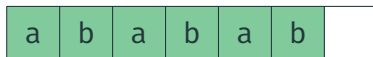
accord déjà constaté auparavant sur 4 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



↕ Concordance : p ne bouge pas



accord déjà constaté auparavant sur 5 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$

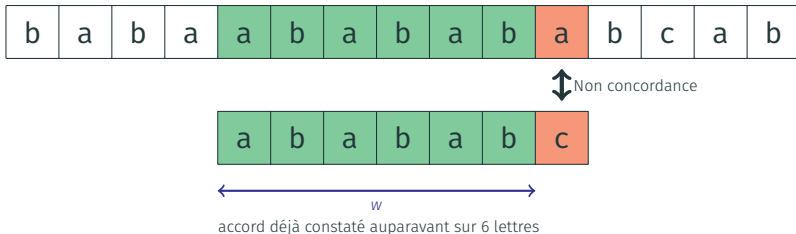


Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcb}$

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

On décale p de $|w| - s(|w|) = 6 - 4 = 2$.

a	b	a	b	a	b	c
---	---	---	---	---	---	---



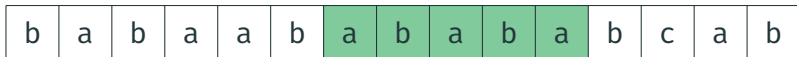
accord déjà constaté auparavant sur 6 lettres

a	b	a	b			
---	---	---	---	--	--	--

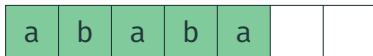
On capitalise 4 comparaison antérieure

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



↕ Concordance

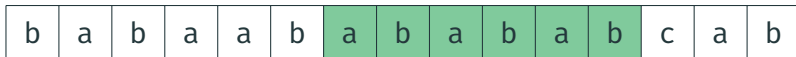


w

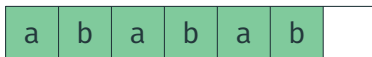
accord déjà constaté auparavant sur 4 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



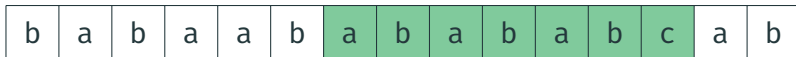
↕ Concordance



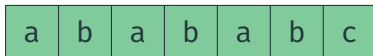
accord déjà constaté auparavant sur 5 lettres

Illustration : amélioration de Morris Pratt

Recherche du motif $p = \text{abababc}$ dans la chaîne
 $s = \text{babaababababcab}$



↕ Concordance



w

accord déjà constaté auparavant sur 6 lettres

Bord compatible

Définition

Soient w un préfixe propre d'un mot p et b un bord de w . On note x la lettre suivant w dans p et y la lettre suivant b dans w , i.e.

$$p = wx \dots \quad \text{et} \quad w = by \dots$$

La bord b est **compatible** si $x \neq y$.

Exemple

Le mot $w = \mathbf{aba}$, préfixe de $p = \mathbf{abababc}$, n'a pas de bord compatible
le mot $w = \mathbf{ababab}$, préfixe de $p = \mathbf{abababc}$, a pour ensemble de bords compatible $\{\mathbf{abab}, \mathbf{ab}, \varepsilon\}$.

Calcul du plus long bord d'un mot

Soit $w = x_0x_1 \cdots x_{i-1}$ et x_i la lettre suivante dans p . On note $r(i)$ la longueur du plus long bord compatible de w . On pose $r(i) = -1$ s'il n'y a pas de bord compatible.

$$r(i) = \begin{cases} s(i) & \text{si } x_{s(i)} \neq x_i \\ r(s(i)) & \text{sinon} \end{cases}$$

Par convention, on pose aussi $r(0) = -1$.

Calcul du bord compatible d'un mot : exemple

Avec $p = \text{abababc}$

i	0	1	2	3	4	5	6
w	ϵ	a	ab	aba	abab	ababa	ababab
succ. w ds p	a	b	a	b	a	b	c
bord compat. b	$\perp$	ϵ	$\perp$	ϵ	$\perp$	ϵ	abab
succ. b ds w		a		a		a	a
$r(i)$	-1	0	-1	0	-1	0	4

Implémentation python (plus long bord compatible)

```
def pluslongbordcomp(p):  
    r = [0] * len(p)  
    r[0] = i = -1  
    for j in range(1, len(p)):  
        while i >= 0 and p[i] != p[j-1]:  
            i = r[i]  
        i += 1  
        if p[i] != p[j]:  
            r[j] = i  
        else:  
            r[j] = r[i]  
    return r
```

Analyse de la complexité

Avec $m = |p|$, nous observons les faits suivants :

- La valeur de i est toujours ≥ -1 .
- La valeur de i ne peut s'incrémenter qu'une seule fois par passage dans la boucle **for**.
- La valeur de i décroît strictement à chaque passage dans la boucle **while**.

Il y a m incréments, donc au plus m étapes de décroissance de i possibles.

La complexité est en $O(m)$.

Seconde amélioration : Knuth Morris Pratt

~ 1970



Donald Ervin Knuth
(1938)

Principe

On fait glisser le motif p le long de la chaîne s :

- On compare p à s caractère par caractère.
- En cas d'échec, on se recale sur le plus grand bord compatible.

Autrement dit, si w est le préfixe de p qui a été reconnu, on se décale de $|w| - r(w)$.

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$$r(0) = -1$$

a						
---	--	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a						
---	--	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b					
---	---	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a				
---	---	---	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b			
---	---	---	---	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$$r(3) = 0$$

a	b	a	b			
---	---	---	---	--	--	--

$$\text{bord compatible}(\text{aba}) = \varepsilon$$

--	--	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a						
---	--	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = abababc$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b					
---	---	--	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a				
---	---	---	--	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = abababc$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b			
---	---	---	---	--	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = abababc$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b	a		
---	---	---	---	---	--	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = abababc$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b	a	b	
---	---	---	---	---	---	--

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = abababc$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	b	a	b	a	b	c
---	---	---	---	---	---	---

Illustration : amélioration de Knuth Morris Pratt

Recherche du motif $p = \text{abababc}$.

b	a	b	a	a	b	a	b	a	b	a	b	c	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$$r(6) = 4$$

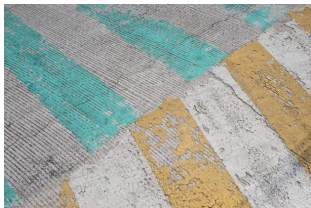
a	b	a	b	a	b	c
---	---	---	---	---	---	---

bord compat. (ababab) = abab

a	b	a	b	a	b	c
---	---	---	---	---	---	---

Implémentation de Knuth Morris Pratt

```
def knuthMorrisPratt(s, p):  
    r = pluslongbordcomp(p)  
    i = j = 0  
    while i < len(p) and j < len(s):  
        if p[i] == s[j]:  
            i += 1  
            j += 1  
        else:  
            if r[i] == -1:  
                i = 0  
                j += 1  
            else:  
                i = r[i]  
    if i == len(p):  
        return j-i  
    else:  
        return -1
```



Analyse de la complexité

Avec $n = |s|$ et $m = |p|$, nous observons les faits suivants :

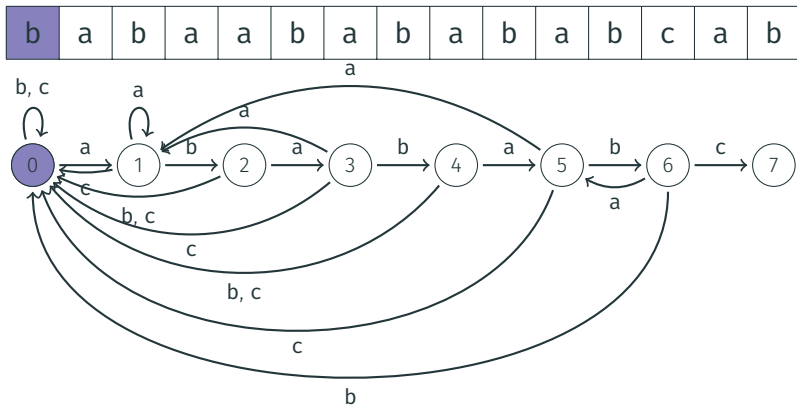
- La valeur de j parcourt l'intervalle $\llbracket 0, n \rrbracket$ par incréments successifs.
- La valeur de i est toujours ≥ 0 (indice dans le motif p).
- La valeur de i ne peut s'incrémenter qu'une seule fois par incrément de j .
- A chaque passage dans la boucle **while**, soit j est incrémenté, soit i décroît strictement.

Il y a $\leq n$ incréments de j , donc au plus n étapes de décroissance de i possible.

La complexité est en $O(n)$ pour la boucle, $O(m)$ pour le calcul de r et $O(m + n)$ au total.

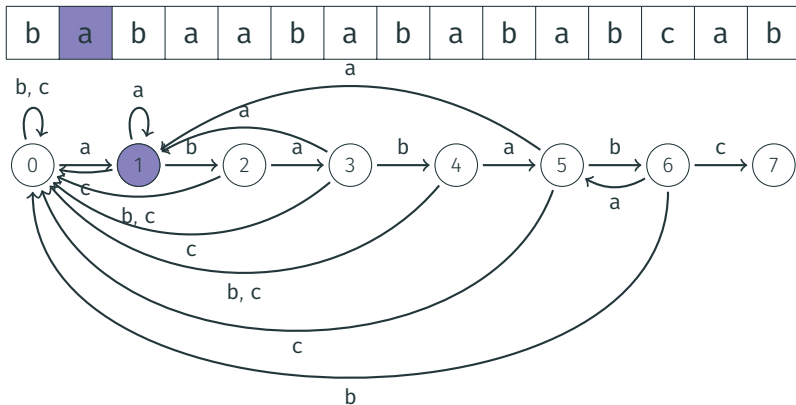
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



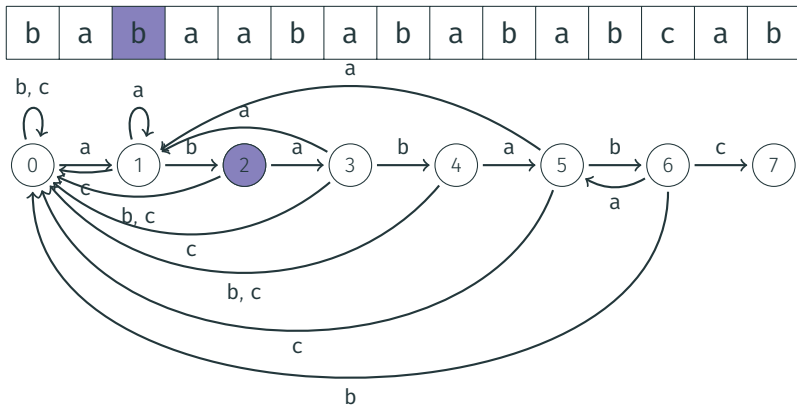
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



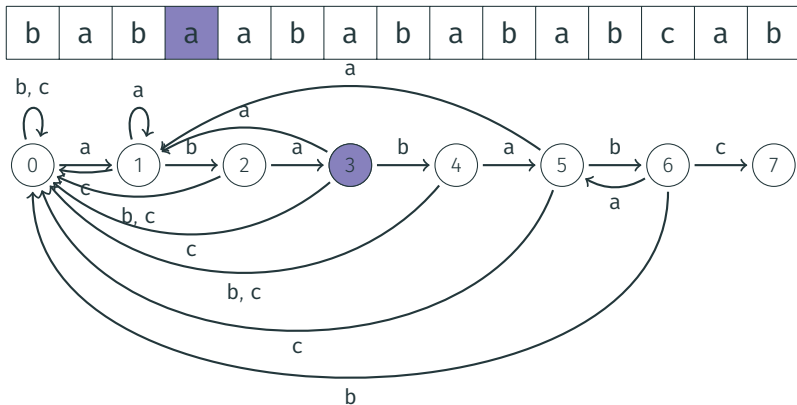
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



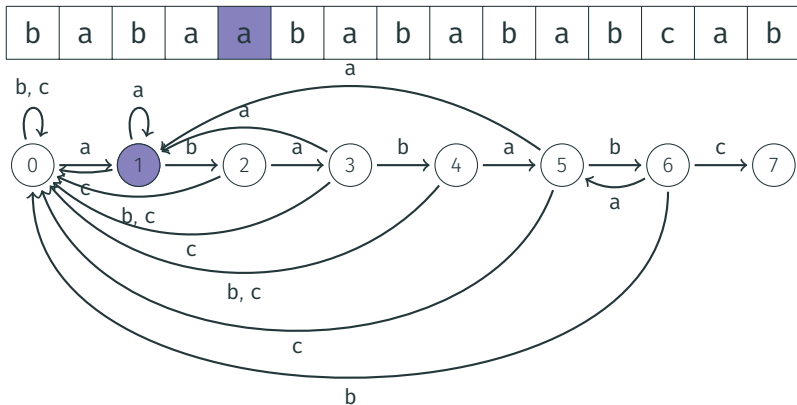
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



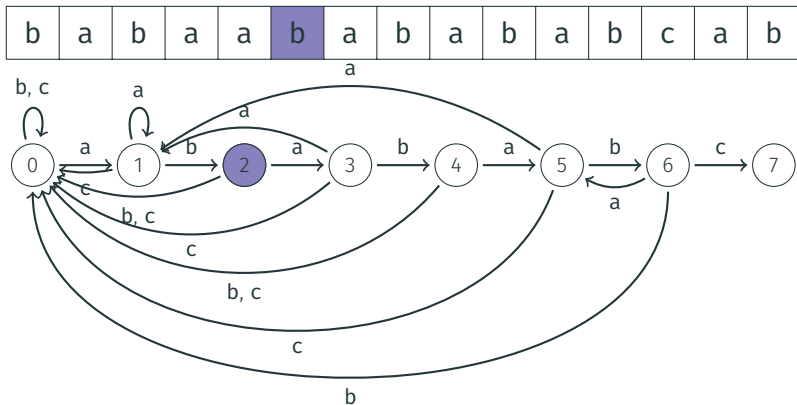
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



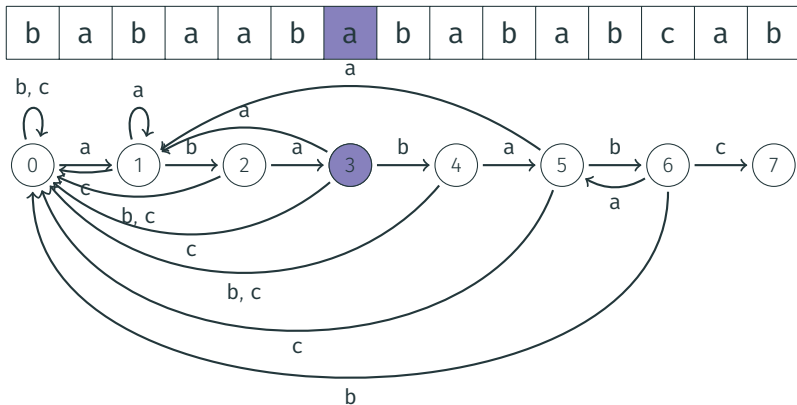
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



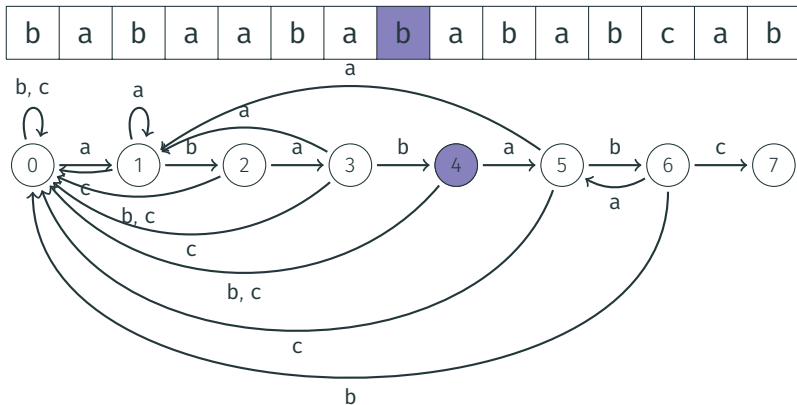
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



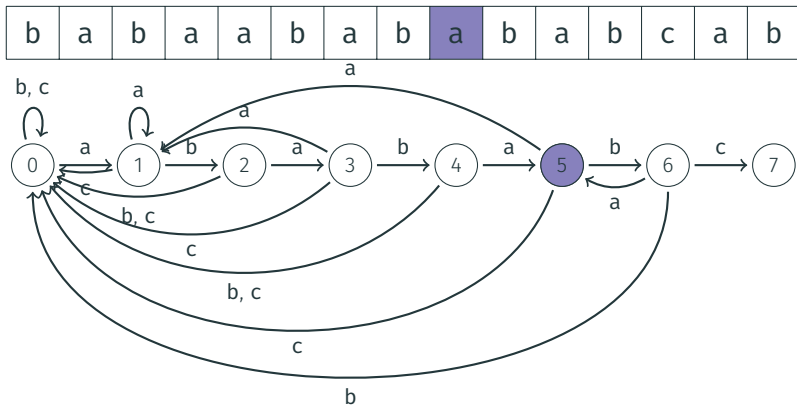
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



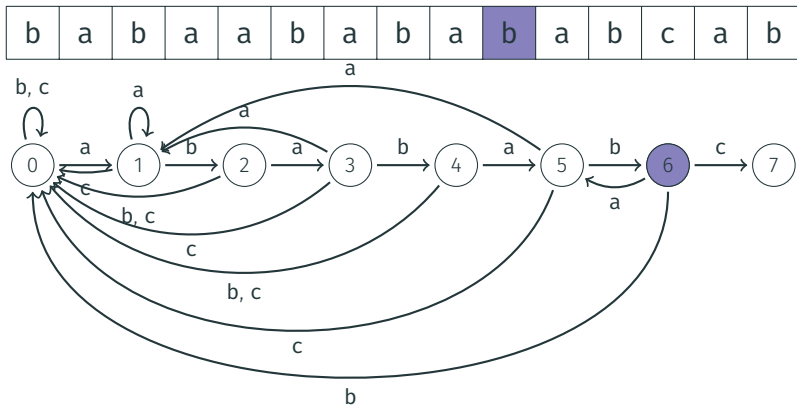
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



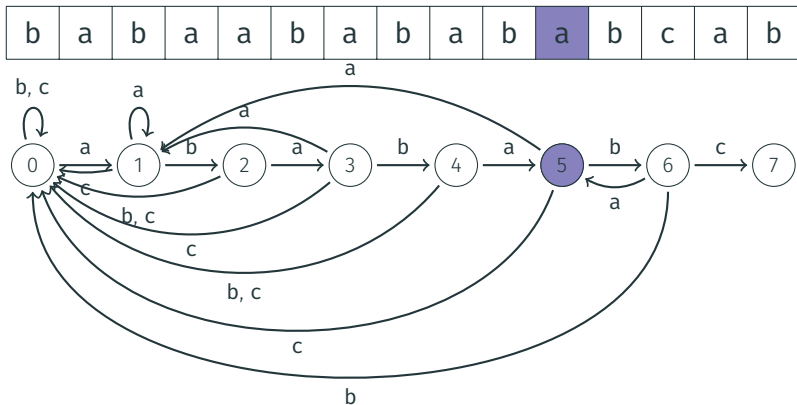
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



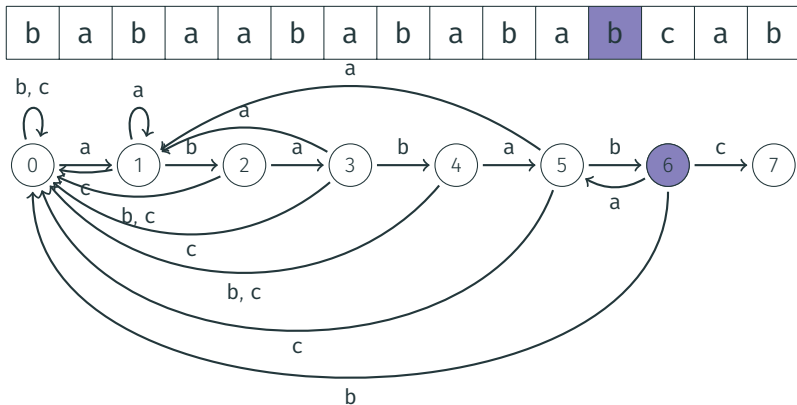
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



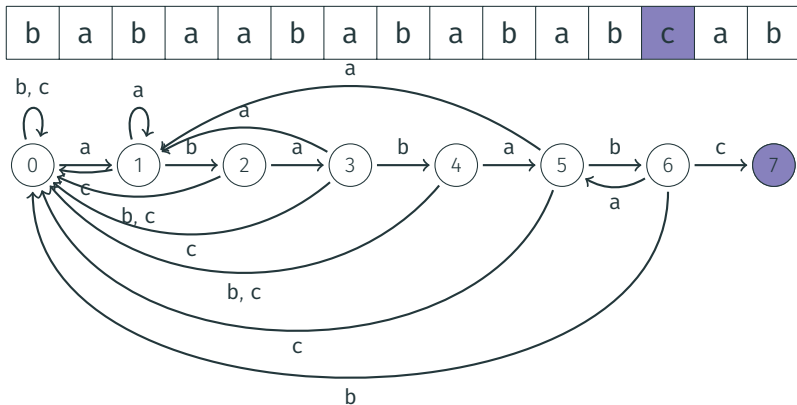
Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



Interprétation graphique

On peut interpréter i comme une position dans un graphe et j comme une position d'une tête de lecture dans la chaîne.



Définition

Un **algorithme en ligne** est un algorithme qui reçoit ses données d'entrée sous forme d'un flux et qui doit prendre des décisions au fur et à mesure sans connaître l'argument d'entrée dans sa totalité.

Exemple

Le motif p étant fixé, la recherche de p comme facteur du mot s par l'**algorithme de KMP** est un algorithme en ligne.

Reconnaître une famille de motifs

La structure de trie

Définition

Soit Σ un alphabet de cardinal σ . Un **trie** (ou **arbre préfixe**) est un arbre étiqueté dont les nœuds internes sont d'arité σ .

La structure de trie

Définition

Soit Σ un alphabet de cardinal σ . Un **trie** (ou **arbre préfixe**) est un arbre étiqueté dont les nœuds internes sont d'arité σ .

Propriété

Un trie réalise le type abstrait **dictionnaire** dans lequel les clés sont des mots de Σ^* et les valeurs sont les étiquettes.

La structure de trie

Définition

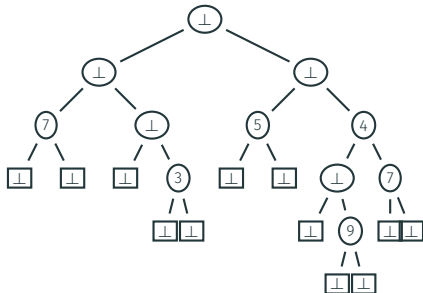
Soit Σ un alphabet de cardinal σ . Un **trie** (ou **arbre préfixe**) est un arbre étiqueté dont les nœuds internes sont d'arité σ .

Propriété

Un trie réalise le type abstrait **dictionnaire** dans lequel les clés sont des mots de Σ^* et les valeurs sont les étiquettes.

Exemple

Sur l'alphabet binaire
 $\Sigma = \{a, b\}$, le dictionnaire
 $\{aa \mapsto 7, abb \mapsto 3, ba \mapsto 5, bb \mapsto 4, , bbab \mapsto 9, bbb \mapsto 7\}$ se représente par

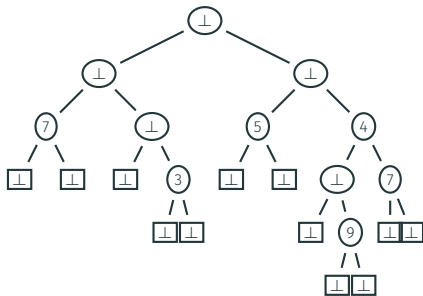


Fonctionnement d'un trie

La valeur de clé $x_0x_1 \cdots x_{k-1} \in \Sigma^*$ est l'étiquette du sommet obtenu par un parcours dans l'arbre depuis la racine en considérant le sous-arbre d'indice x_i à l'étape i .

Exemple

Pour trouver la valeur de clé **abb**, on suit le chemin

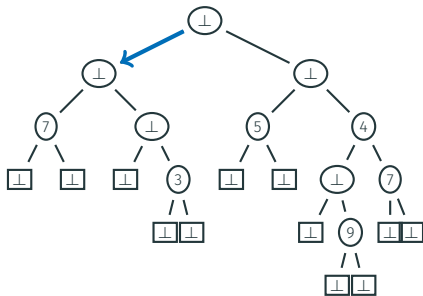


Fonctionnement d'un trie

La valeur de clé $x_0x_1 \cdots x_{k-1} \in \Sigma^*$ est l'étiquette du sommet obtenu par un parcours dans l'arbre depuis la racine en considérant le sous-arbre d'indice x_i à l'étape i .

Exemple

Pour trouver la valeur de clé **abb**, on suit le chemin

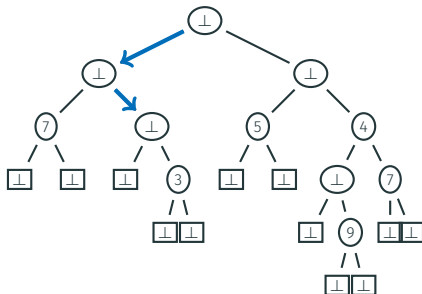


Fonctionnement d'un trie

La valeur de clé $x_0x_1 \cdots x_{k-1} \in \Sigma^*$ est l'étiquette du sommet obtenu par un parcours dans l'arbre depuis la racine en considérant le sous-arbre d'indice x_i à l'étape i .

Exemple

Pour trouver la valeur de clé **abb**, on suit le chemin



Fonctionnement d'un trie

La valeur de clé $x_0x_1 \cdots x_{k-1} \in \Sigma^*$ est l'étiquette du sommet obtenu par un parcours dans l'arbre depuis la racine en considérant le sous-arbre d'indice x_i à l'étape i .

Exemple

Pour trouver la valeur de clé **abb**, on suit le chemin

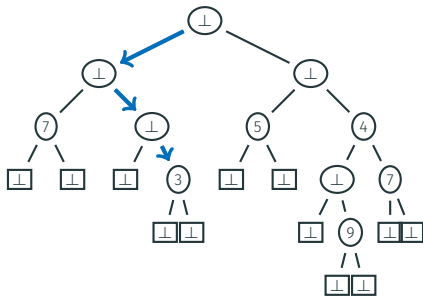


Illustration d'un trie - stockage concis

Principe

Lorsque l'alphabet Σ est grand, on ne stocke que les fils non-vide.

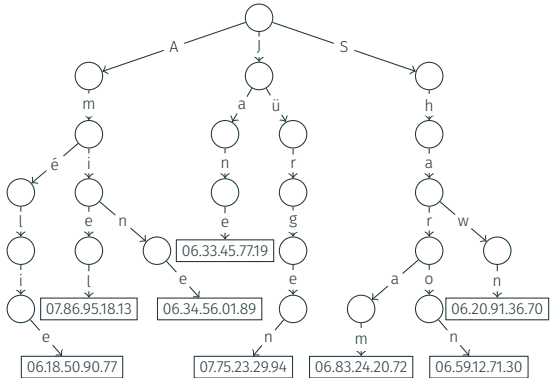
Illustration d'un trie - stockage concis

Principe

Lorsque l'alphabet Σ est grand, on ne stocke que les fils non-vide.

Exemple (Arbre préfixe ou *Trie*)

Clé	Valeur
Amélie	06.18.50.90.77
Amiel	07.86.95.18.13
Amine	06.34.56.01.89
Jane	06.33.45.77.19
Jürgen	07.75.23.29.94
Sharam	06.83.24.20.72
Sharon	06.59.12.71.30
Shawn	06.20.91.36.70



Implémentation en Python

Principe

Lorsque l'alphabet Σ est grand, on ne stocke que les fils non-vide. Ici, on utilise un dictionnaire qui associe à chaque lettre le « trie fils ».

```
class Trie():
    def __init__(self):
        self.children = dict()
        self.value = None
    def __setitem__(self, key, value):
        if key == "":
            self.value = value
        else:
            x = key[0]
            if x not in self.children:
                self.children[x] = Trie()
            self.children[x].__setitem__(key[1:], value)
    def __getitem__(self, key):
        if key == "":
            return(self.value)
        else:
            x = key[0]
            if x not in self.children:
                return None
            else:
                return self.children[x].__getitem__(key[1:])
```

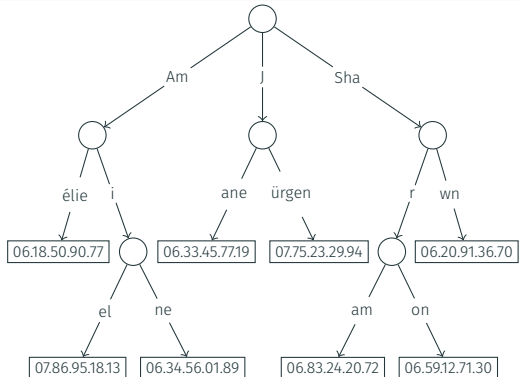
Variante : les arbres Patricia

Principe

Lorsque le trie est creux (beaucoup d'arêtes et peu d'associations), on regroupe plusieurs arêtes formant un chemin.

Exemple (Arbre radix ou arbre Patricia, 1968)

Clé	Valeur
Amélie	06.18.50.90.77
Amiel	07.86.95.18.13
Amine	06.34.56.01.89
Jane	06.33.45.77.19
Jürgen	07.75.23.29.94
Sharam	06.83.24.20.72
Sharon	06.59.12.71.30
Shawn	06.20.91.36.70



Problème de recherche de plusieurs sous-chaînes

Étant donné un mot s et une famille finie de mots $\{p_1, p_2, \dots, p_k\}$ sur un alphabet Σ , le problème de la **recherche de plusieurs sous-chaînes** consiste à déterminer si l'un des motifs p_i est un *facteur* de s et, éventuellement, localiser p_i dans s (voire toutes les occurrences de tous les p_i).

Applications

Recherche de signature de virus informatique ou de séquence particulière dans un génome.

L'algorithme d'Aho Corasick

Précalcul :

1. Construire le trie contenant les mots $\{p_1, p_2, \dots, p_k\}$
2. Construire un lien entre tout sommet du trie et le plus long suffixe existant dans le trie.

Recherche de motif :

1. Lire chaque lettre de s
2. Si possible, progresser dans le trie, sinon, utiliser les liens secondaires pour se replacer sur le plus long suffixe.

Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie.

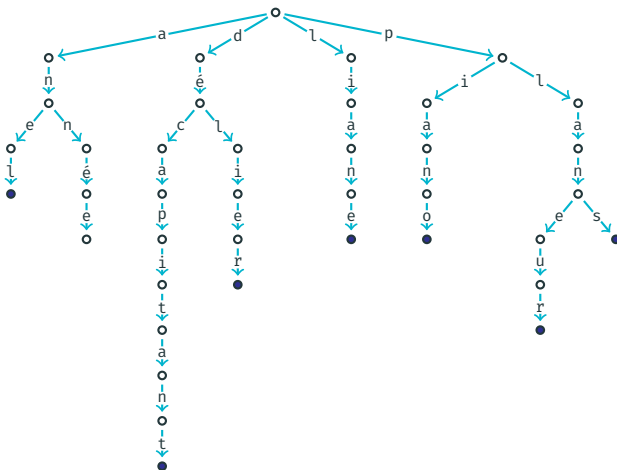


Illustration du précalcul d'Aho Corasick

Recherche des motifs **an**l, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans**.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

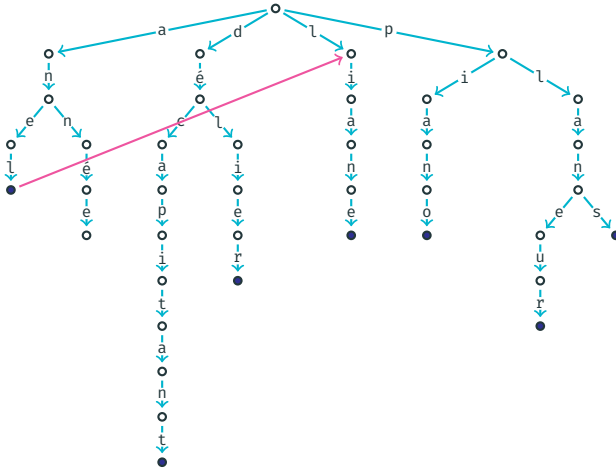


Illustration du précalcul d'Aho Corasick

Recherche des motifs **an**l, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans**.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

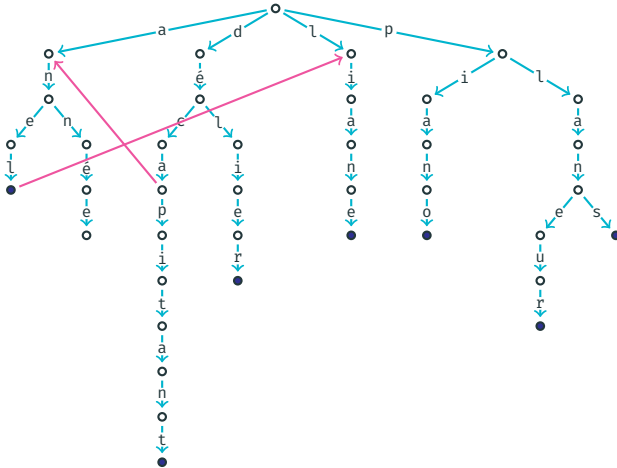


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

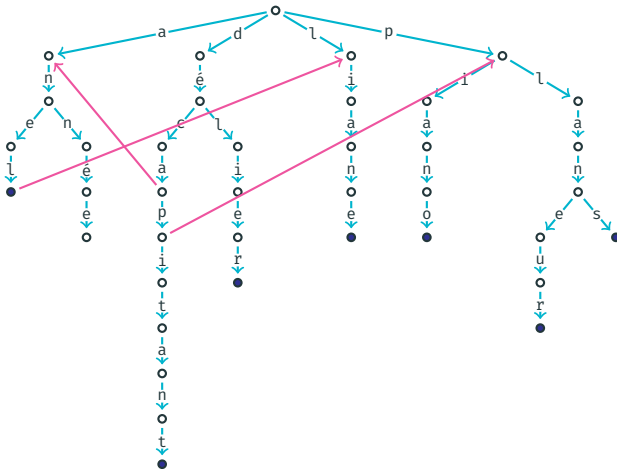


Illustration du précalcul d'Aho Corasick

Recherche des motifs **an**el, année, décapitant, délier, liane, piano, planeur, plans.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

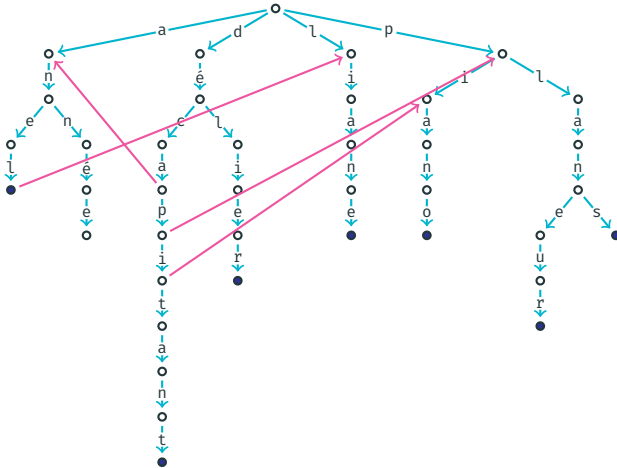


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

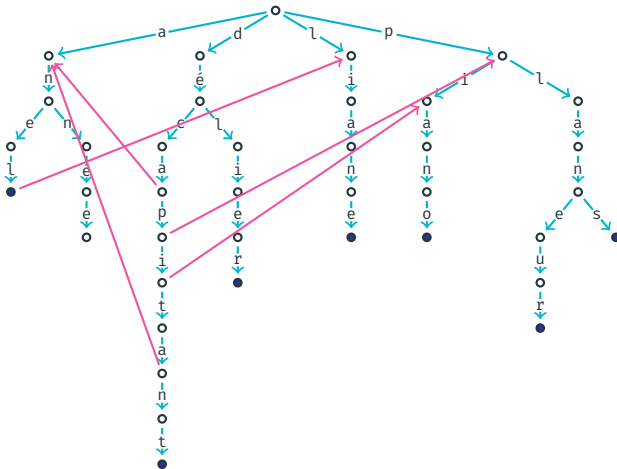


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

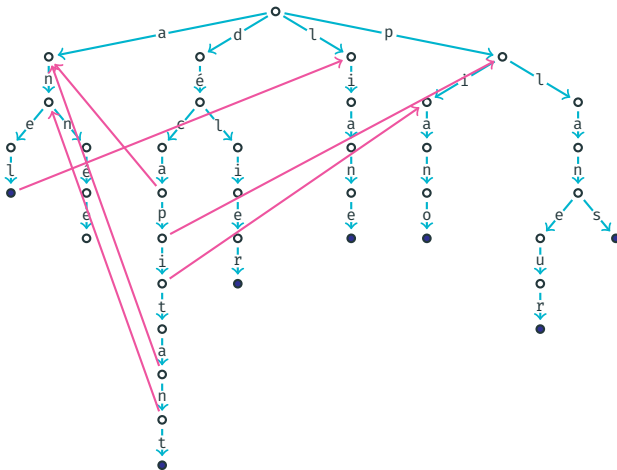


Illustration du précalcul d'Aho Corasick

Recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans**.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

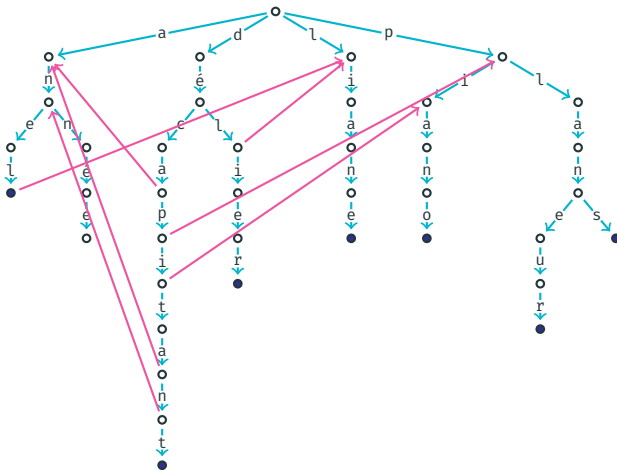


Illustration du précalcul d'Aho Corasick

Recherche des motifs anel, année, décapitant, délier, liane, piano, planeur, plans.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

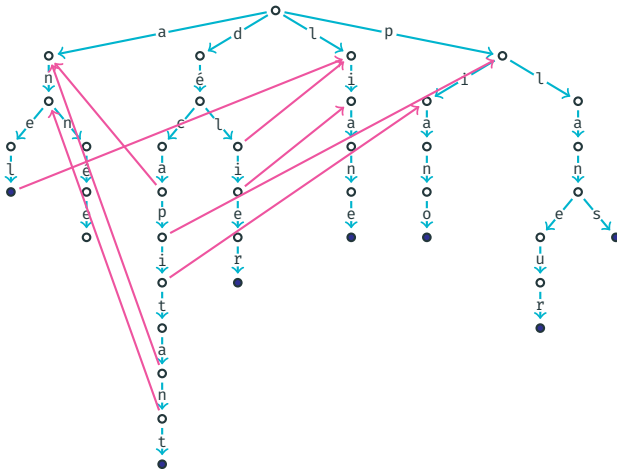


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

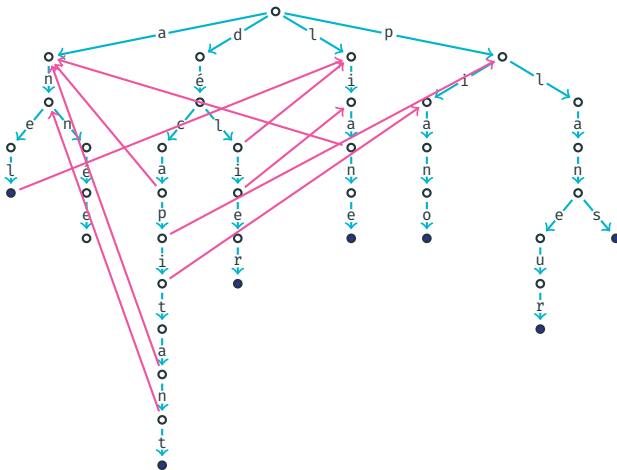


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

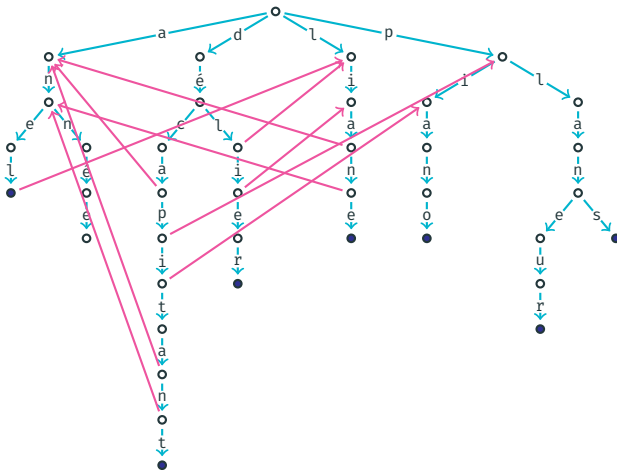


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

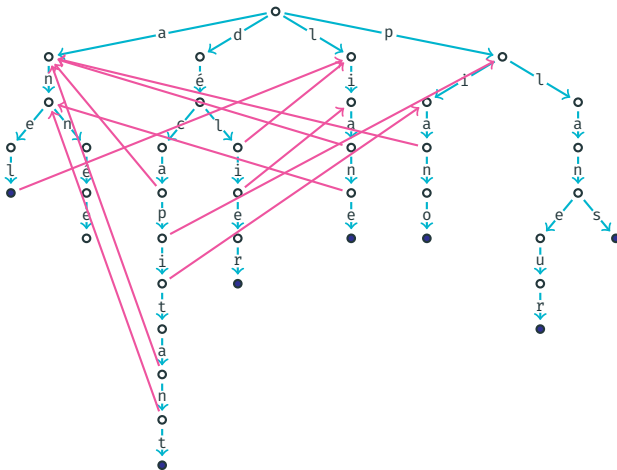


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

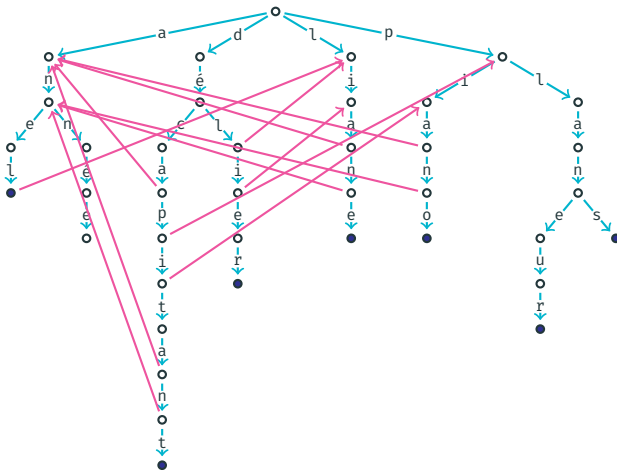


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

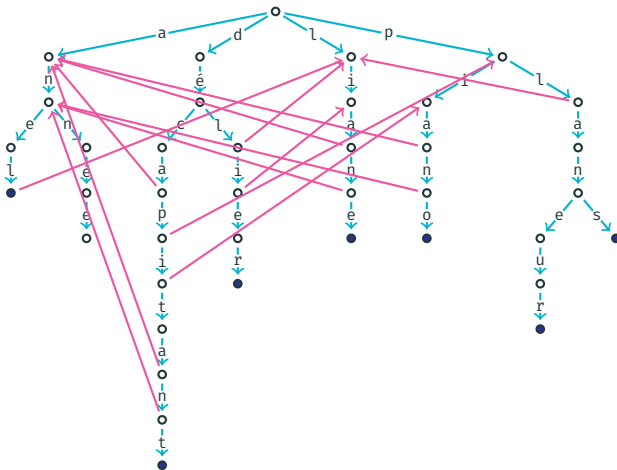


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

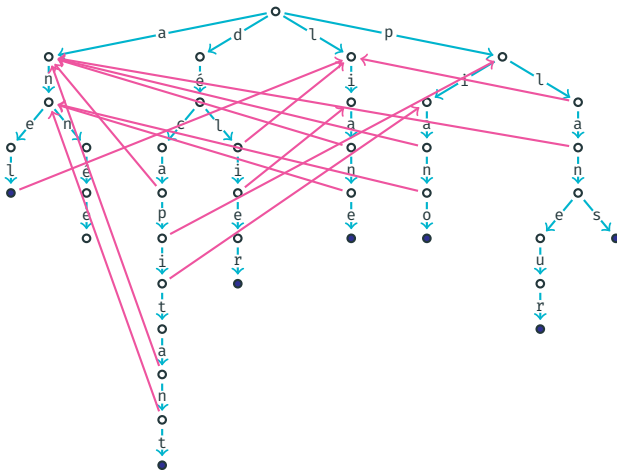


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

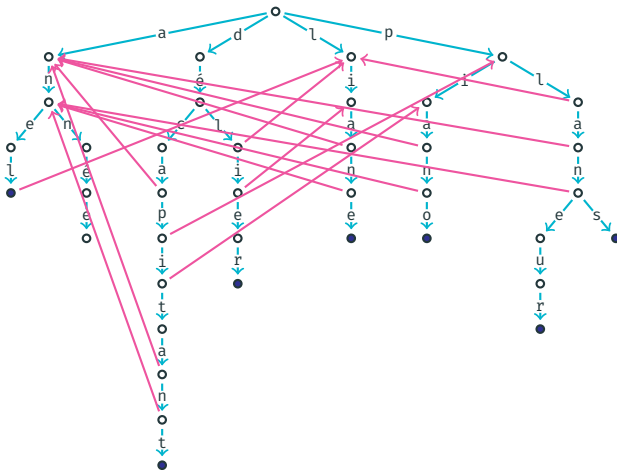


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

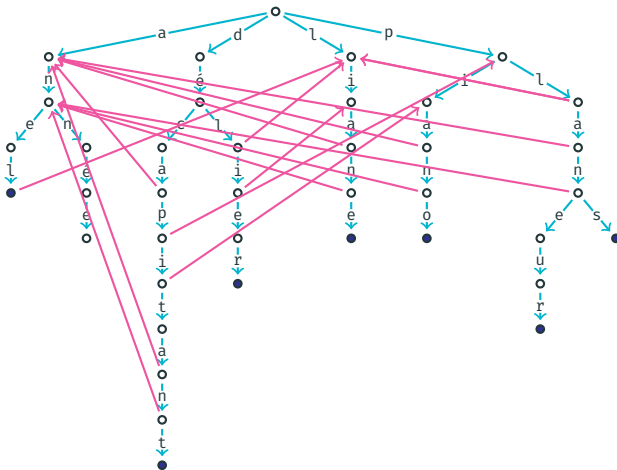


Illustration du précalcul d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans`.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

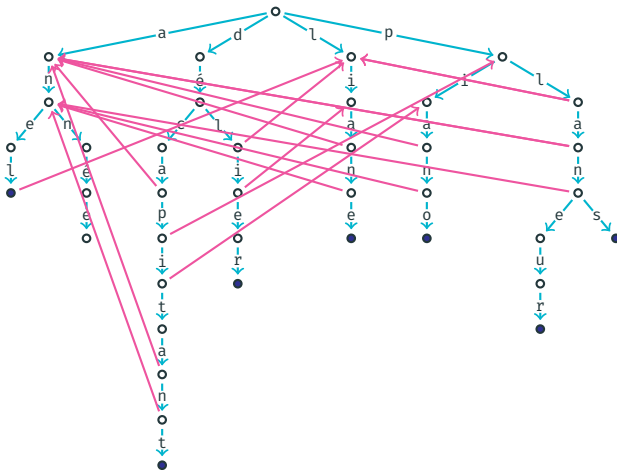


Illustration du précalcul d'Aho Corasick

Recherche des motifs **an**el, année, **dé**capitant, **dé**lier, liane, piano, planeur, plans.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

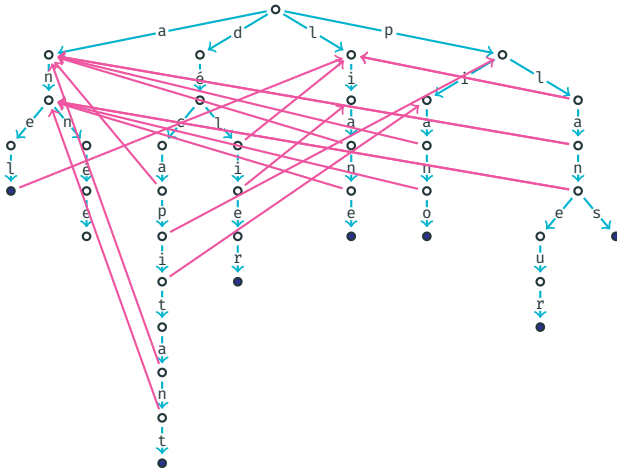


Illustration du précalcul d'Aho Corasick

Recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans**.
On commence par construire un trie. On rajoute les liaisons vers les suffixes présents dans le trie (on ne représente pas les liaisons qui pointent vers la racine)

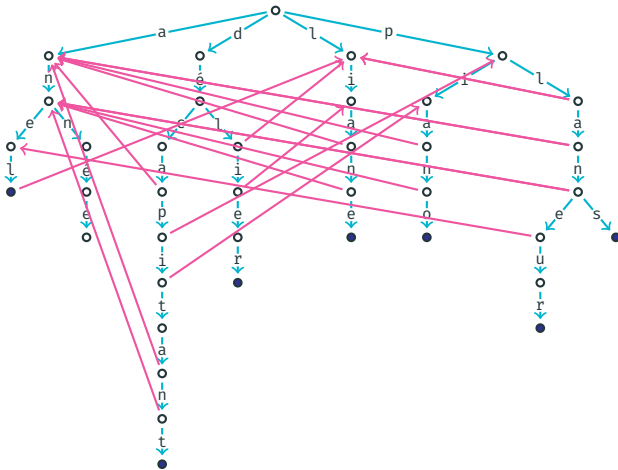


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

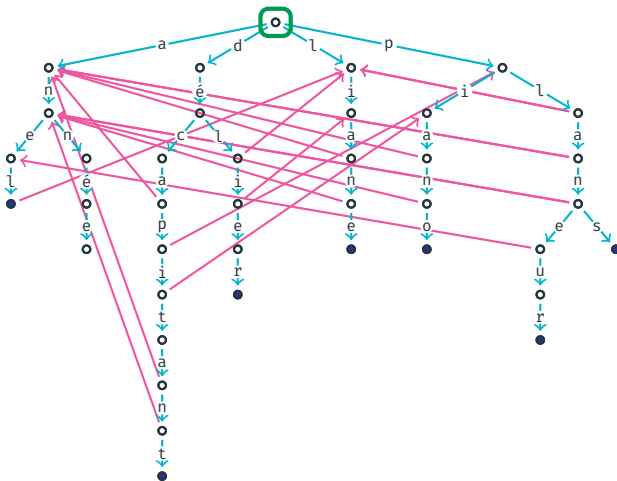


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

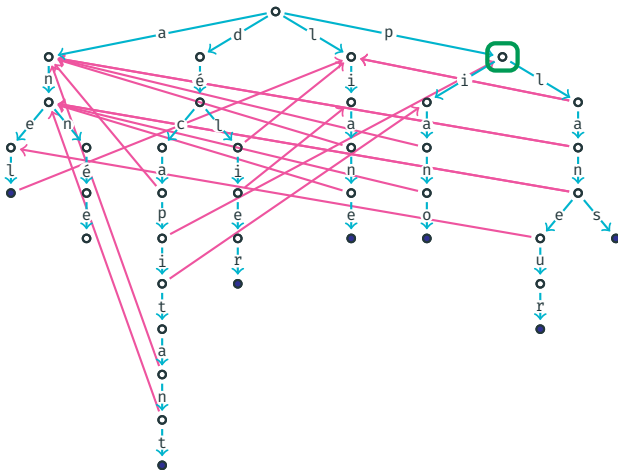


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

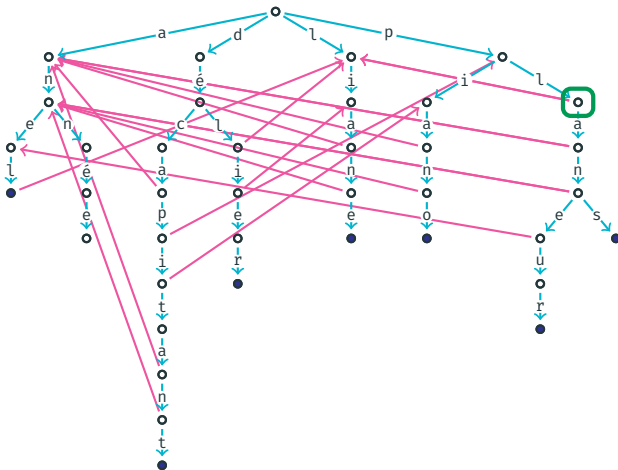


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeleannée!`

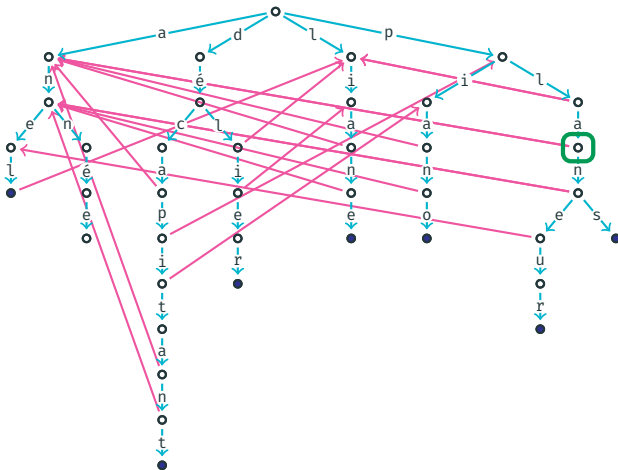


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

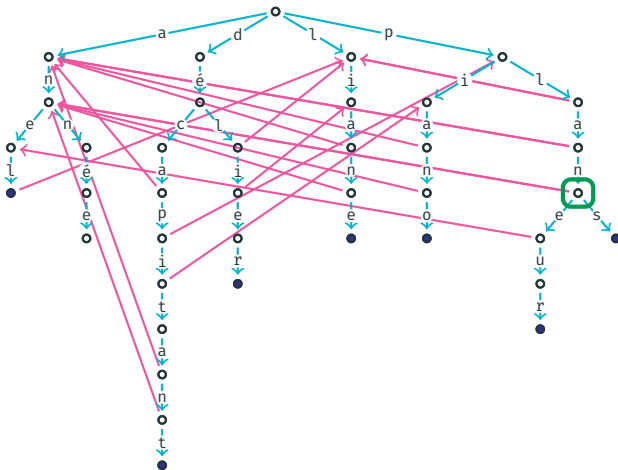


Illustration de la recherche d'Aho Corasick

Recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans** dans la chaîne

s = plandéliredelannée!

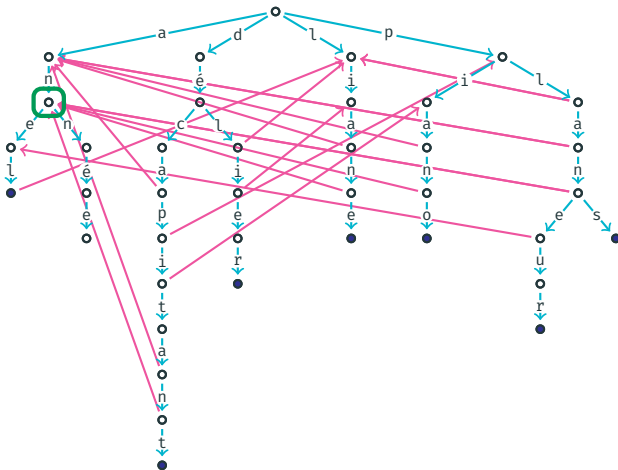


Illustration de la recherche d'Aho Corasick

Recherche des motifs anel, année, décapitant, délier, liane, piano, planeur, plans dans la chaîne

s = plandéliredelannée!

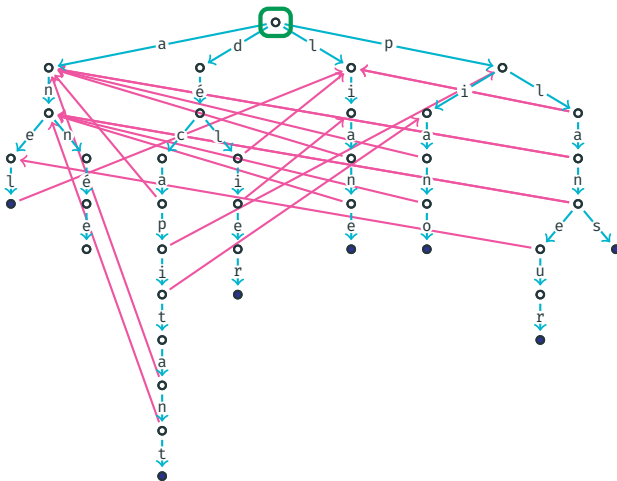


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

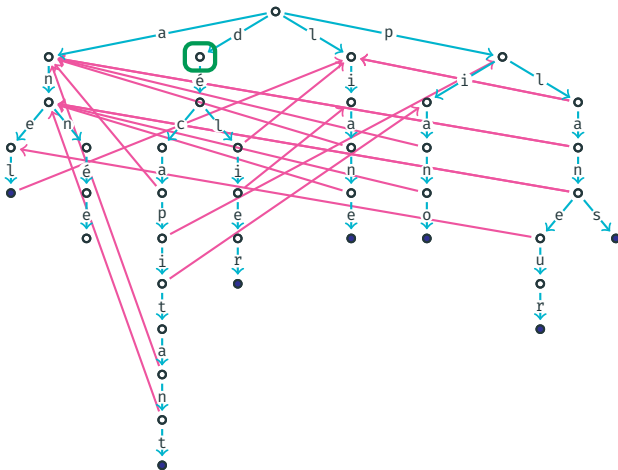


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

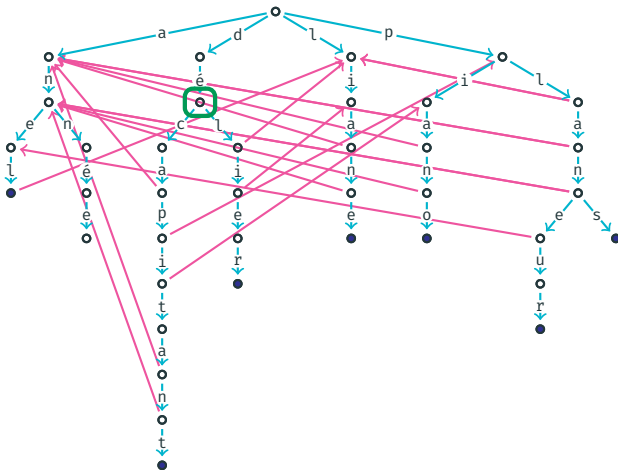


Illustration de la recherche d'Aho Corasick

Recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans** dans la chaîne

s = plandéliredelannée!

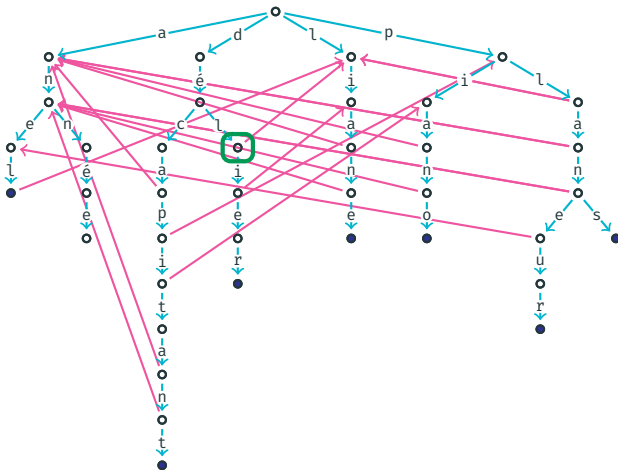


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeleannée!`

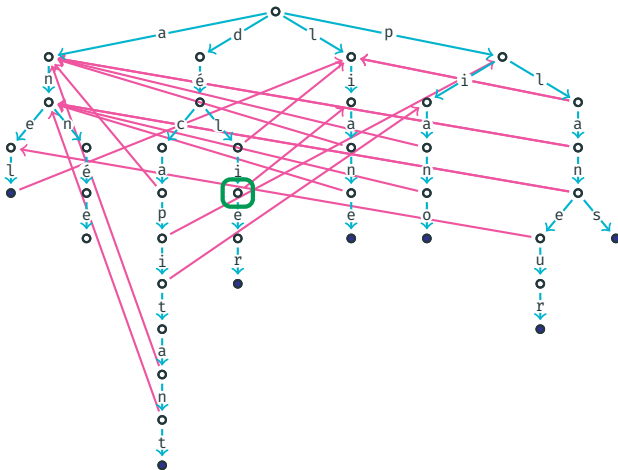


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeleannée!`

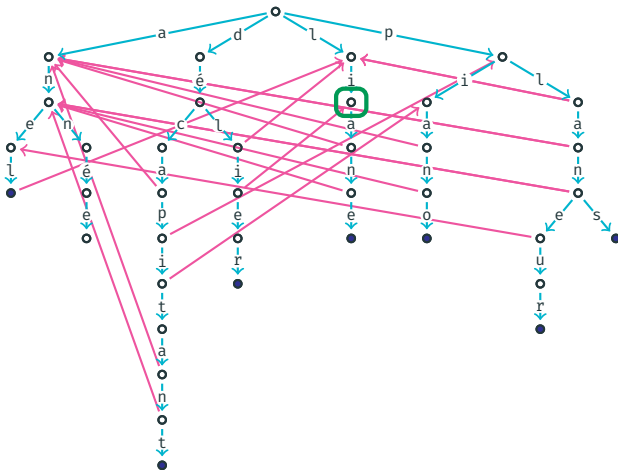


Illustration de la recherche d'Aho Corasick

Recherche des motifs anel, année, décapitant, délier, liane, piano, planeur, plans dans la chaîne

s = plandéliredelannée!

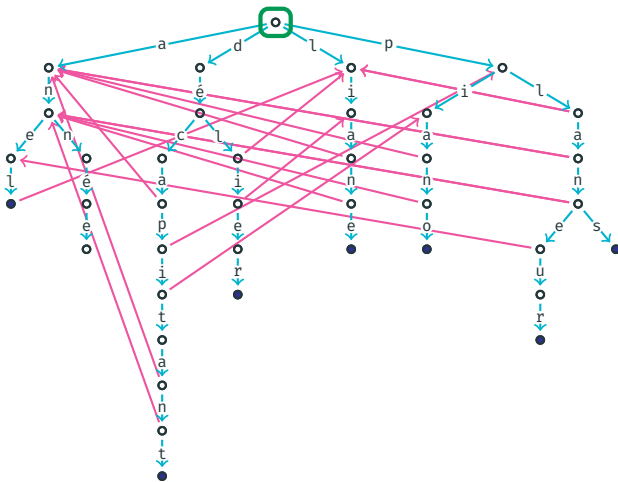


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

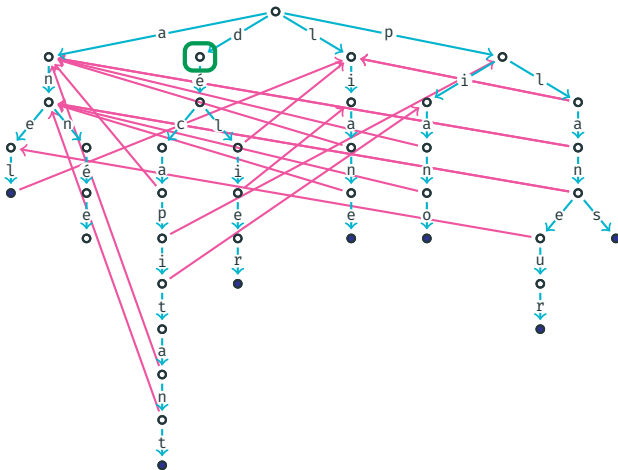


Illustration de la recherche d'Aho Corasick

Recherche des motifs anel, année, décapitant, délier, liane, piano, planeur, plans dans la chaîne

s = plandéliredelannée!

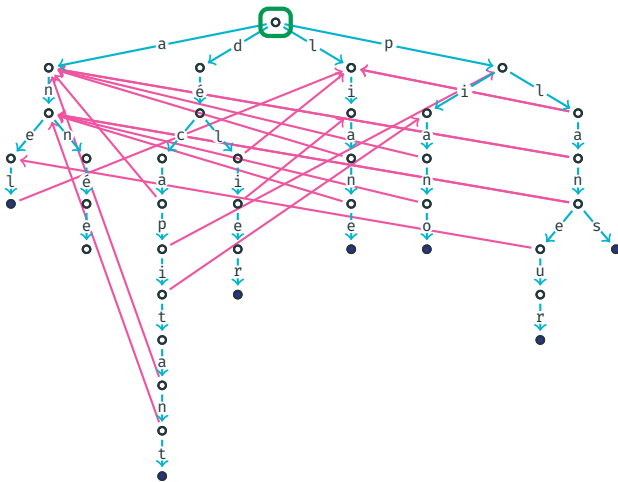


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

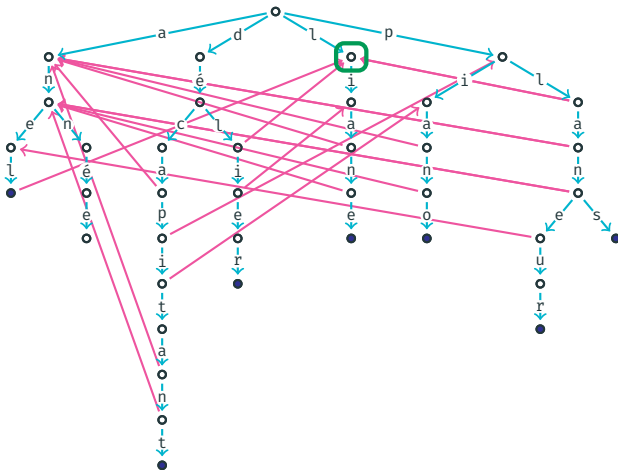


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

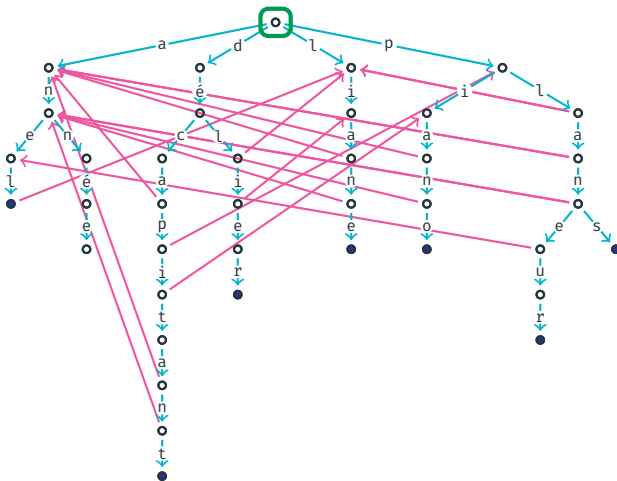


Illustration de la recherche d'Aho Corasick

Recherche des motifs **anel**, **année**, **décapitant**, **délier**, **liane**, **piano**, **planeur**, **plans** dans la chaîne

s = plandéliredelannée!

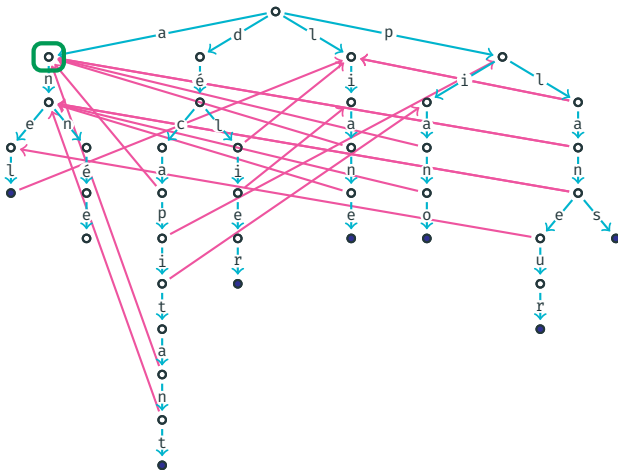


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

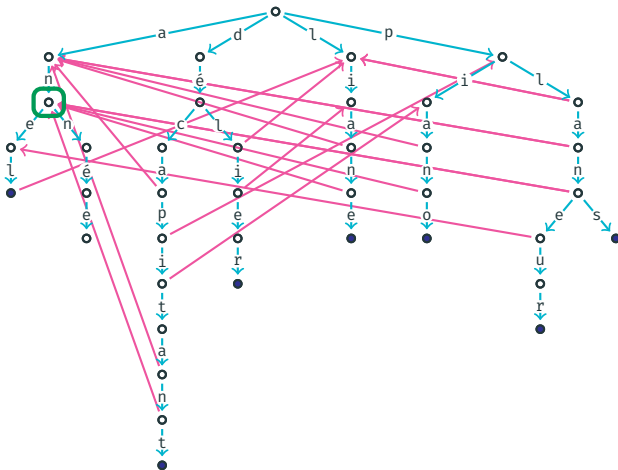


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeleannée!`

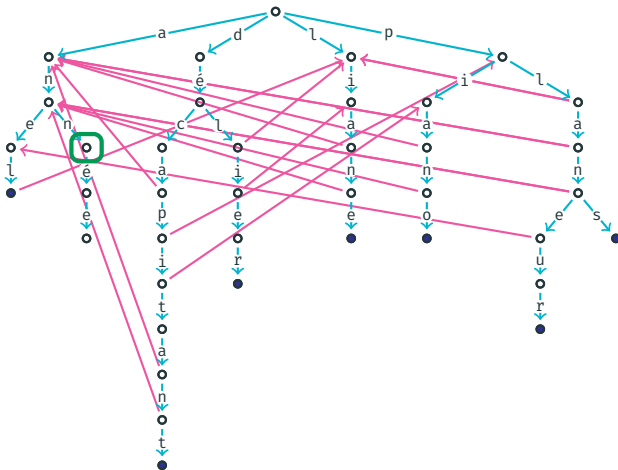


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`

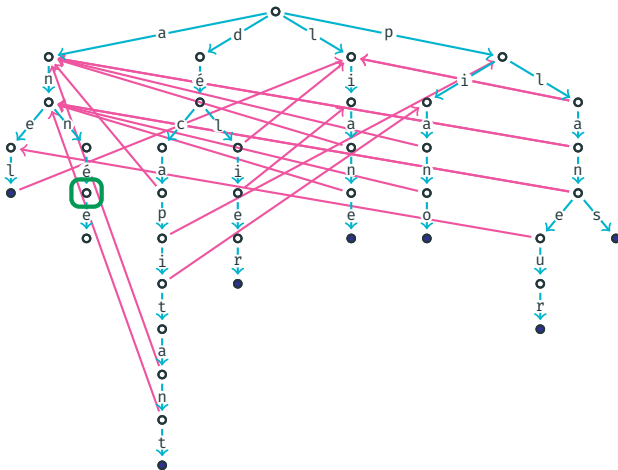
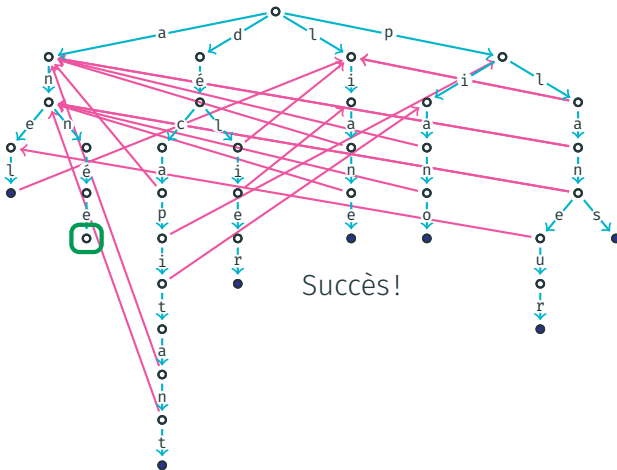


Illustration de la recherche d'Aho Corasick

Recherche des motifs `anel`, `année`, `décapitant`, `délier`, `liane`, `piano`, `planeur`, `plans` dans la chaîne

`s = plandéliredeannée!`



Application au décodage binaire

Définition

Un **code binaire** pour l'alphabet Σ est une application injective $E : \Sigma \rightarrow \{0, 1\}^+$.

Exemple

Avec $\Sigma = \{a, b, c, d, e, f\}$, on fixe le code $E_{\clubsuit}$

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

Définition

Un **codage** via le code binaire E est l'application $E : \Sigma^* \rightarrow \{0, 1\}^*$ définie par

$$E(x_1x_2 \dots x_n) = E(x_1)E(x_2) \dots E(x_n).$$

Définition

Un **codage** via le code binaire E est l'application $E : \Sigma^* \rightarrow \{0,1\}^*$ définie par

$$E(x_1x_2 \dots x_n) = E(x_1)E(x_2) \dots E(x_n).$$

Exemple

Avec le code

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

le mot $cafe \in \Sigma^*$ admet pour codage

$$cafe \mapsto \quad \quad \quad \in \{0,1\}^*.$$



Définition

Un **codage** via le code binaire E est l'application $E : \Sigma^* \rightarrow \{0,1\}^*$ définie par

$$E(x_1x_2 \dots x_n) = E(x_1)E(x_2) \dots E(x_n).$$

Exemple

Avec le code

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

le mot $cafe \in \Sigma^*$ admet pour codage

$$cafe \mapsto 010000101100 \in \{0,1\}^*.$$



Définition

Un codage est dit **uniquement décodable** si l'application

$E : \Sigma^* \rightarrow \{0, 1\}^*$ est injective.

Définition

Un codage est dit **uniquement décodable** si l'application

$E : \Sigma^* \rightarrow \{0, 1\}^*$ est injective.

Exemple

Avec le code

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

l'unique mot w ayant

$$w \mapsto 101000010100 \in \{0, 1\}^*$$

comme codage est le mot $w = \quad \in \Sigma^*$.

Définition

Un codage est dit **uniquement décodable** si l'application

$E : \Sigma^* \rightarrow \{0, 1\}^*$ est injective.

Exemple

Avec le code

$$E_{\clubsuit} : \begin{cases} a \mapsto 000 & ; & c \mapsto 010 & ; & e \mapsto 100 \\ b \mapsto 001 & ; & d \mapsto 011 & ; & f \mapsto 101 \end{cases}$$

l'unique mot w ayant

$$w \mapsto 101000010100 \in \{0, 1\}^*$$

comme codage est le mot $w = \text{face} \in \Sigma^*$.

Définition

Un code E est dit **sans préfixe** si pour toutes lettres $x \neq y$, le mot binaire $E(x)$ n'est pas un préfixe du mot binaire $E(y)$.

Remarque

Dans le *trie* d'un code sans préfixe, seules les feuilles sont étiquetées. Ceci fournit un algorithme de décodage linéaire et montre qu'un code sans préfixe est toujours **uniquement décodable**.

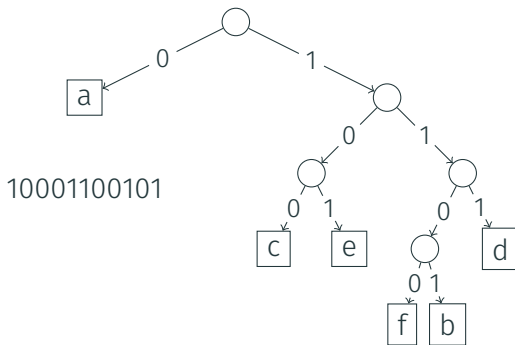
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



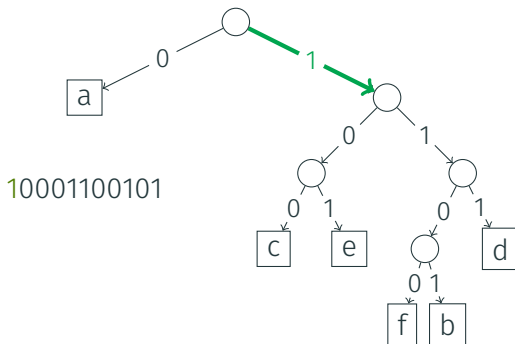
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



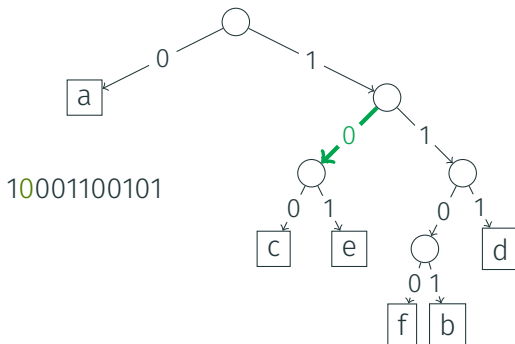
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



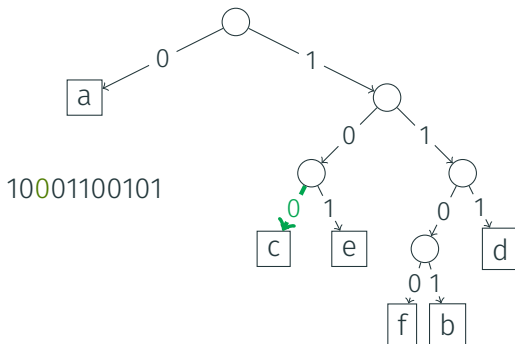
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



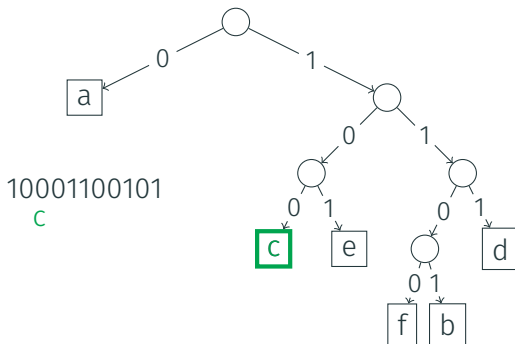
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



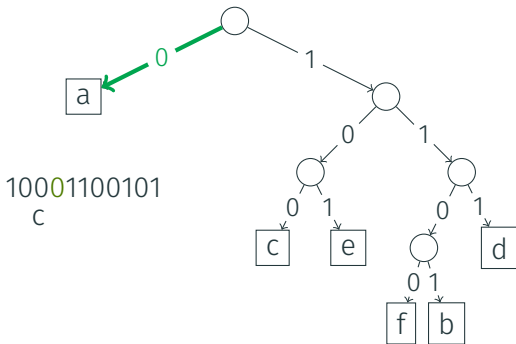
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



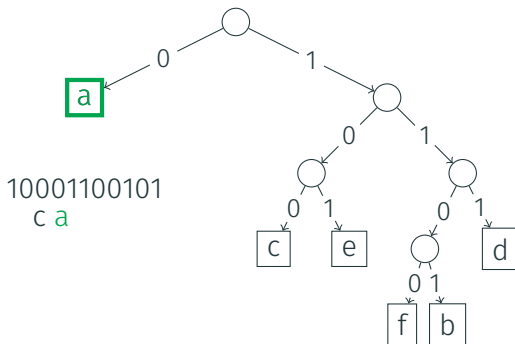
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



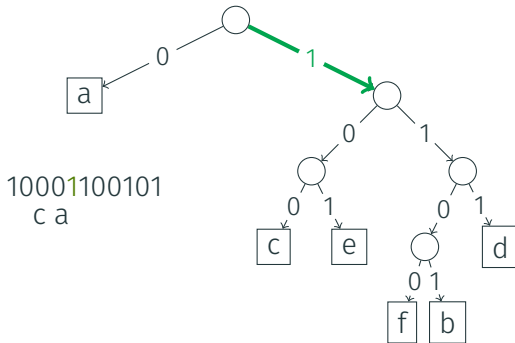
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



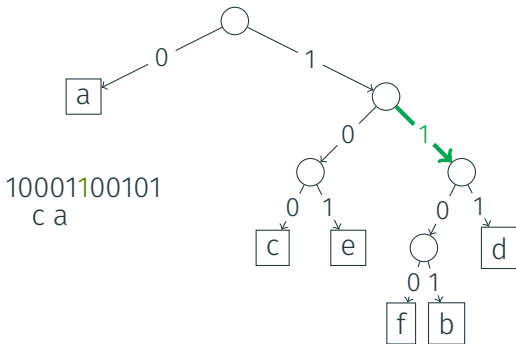
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



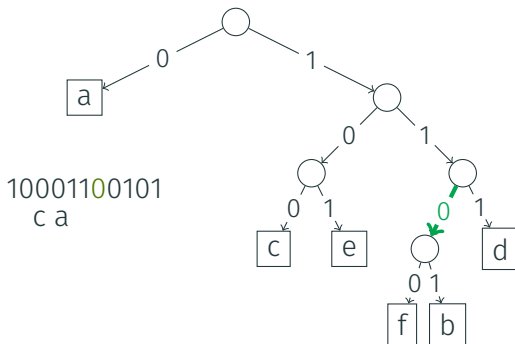
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



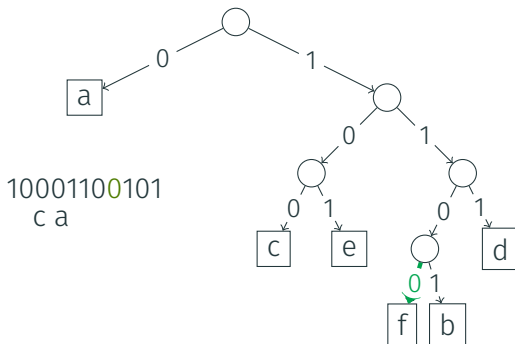
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



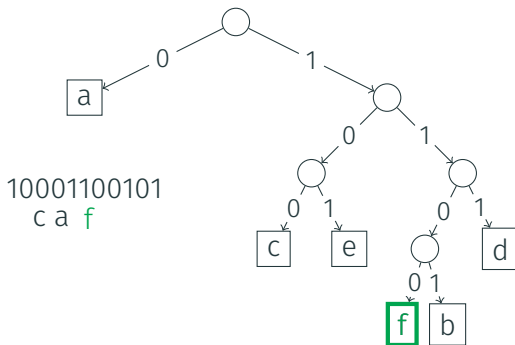
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



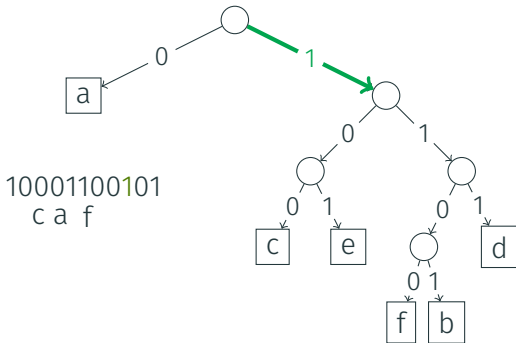
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



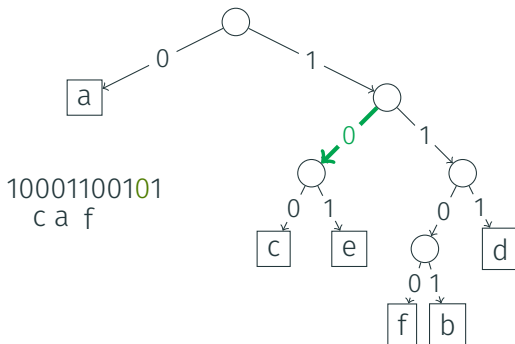
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



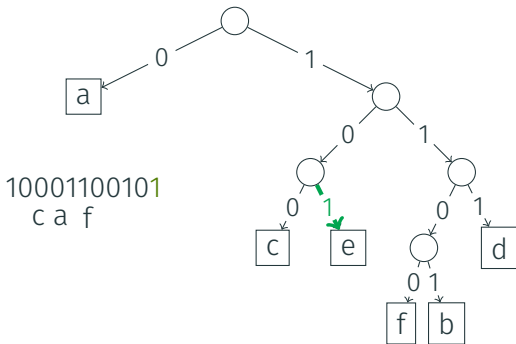
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



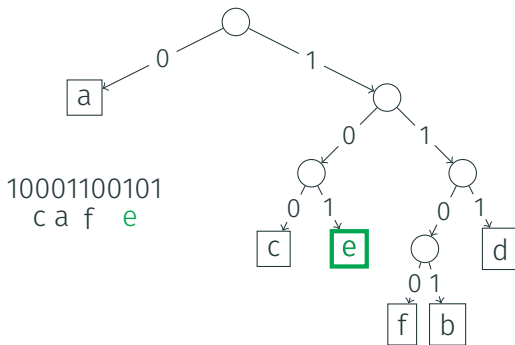
Codes sans préfixes : exemple

Exemple

Le code

$$E_{\spadesuit} : \begin{cases} a \mapsto 0 & ; & c \mapsto 100 & ; & e \mapsto 101 \\ b \mapsto 1101 & ; & d \mapsto 111 & ; & f \mapsto 1100 \end{cases}$$

est sans préfixe et admet comme *trie* l'arbre



Taux de compression

Supposons que l'on encode un mot $w \in \Sigma^*$ par un mot binaire et que l'on connaisse la distribution des occurrences de chaque lettre, on peut calculer **longueur de codage** de w .

Exemple

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	Total
occurrences	45 000	5 000	12 000	16 000	13 000	9 000	100 000
Codage $E_{\clubsuit}$	3	3	3	3	3	3	
Codage $E_{\spadesuit}$	1	4	3	3	3	4	

Taux de compression

Supposons que l'on encode un mot $w \in \Sigma^*$ par un mot binaire et que l'on connaisse la distribution des occurrences de chaque lettre, on peut calculer **longueur de codage** de w .

Exemple

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	Total
occurrences	45 000	5 000	12 000	16 000	13 000	9 000	100 000
Codage $E_{\clubsuit}$	3	3	3	3	3	3	300 000
Codage $E_{\spadesuit}$	1	4	3	3	3	4	

Taux de compression

Supposons que l'on encode un mot $w \in \Sigma^*$ par un mot binaire et que l'on connaisse la distribution des occurrences de chaque lettre, on peut calculer **longueur de codage** de w .

Exemple

	a	b	c	d	e	f	Total
occurrences	45 000	5 000	12 000	16 000	13 000	9 000	100 000
Codage $E_{\clubsuit}$	3	3	3	3	3	3	300 000
Codage $E_{\spadesuit}$	1	4	3	3	3	4	224 000

$$45 + 4 \cdot 5 + 3 \cdot 12 + 3 \cdot 16 + 3 \cdot 13 + 4 \cdot 9 = 224.$$

Compression optimale de données sans perte

Problème

Étant donné des nombres d'occurrences (ou des fréquences) des symboles de Σ , déterminer le code de **longueur de codage** (ou taux de compression) **minimum**.

Compression optimale de données sans perte

Problème

Étant donné des nombres d'occurrences (ou des fréquences) des symboles de Σ , déterminer le code de **longueur de codage** (ou taux de compression) **minimum**.

Théorème (Admis)

La compression optimale de données sans perte est toujours possible en utilisant un code sans préfixes.

Démonstration.

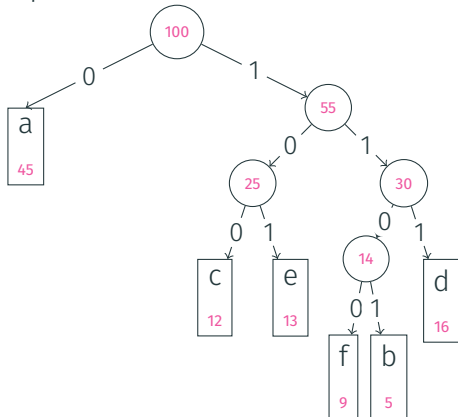
Basé sur l'inégalité de Kraft. $\square$



Arbre de compression

Définition

Étant donné un code $E : \Sigma \rightarrow \{0, 1\}^*$ et des nombres d'occurrence, l'**arbre de compression** de E est le trie pondéré qui représente E où le poids d'une feuille est son nombre d'occurrence et le poids d'un nœud interne est la somme des poids de ses fils.



Étant donné des nombres d'occurrence $(o_\sigma)_{\sigma \in \Sigma}$ des symboles dans un mot $w \in \Sigma^*$ et un arbre de compression A , la **longueur de codage** correspond à

$$L(A) = \sum_{\sigma \in \Sigma} o_\sigma \cdot \text{prof}_A(\sigma).$$

Un arbre de compression est **optimum** s'il minimise $L(A)$.

Lemme

Dans un arbre optimum, tout nœud interne possède deux fils.

Lemme

Dans un arbre optimum, tout nœud interne possède deux fils.

Lemme

Dans un arbre optimum, les deux symboles de plus petite occurrence sont à la profondeur de l'arbre.

Propriétés des arbres de compression optimaux

Lemme

Dans un arbre optimum, tout nœud interne possède deux fils.

Lemme

Dans un arbre optimum, les deux symboles de plus petite occurrence sont à la profondeur de l'arbre.

Lemme

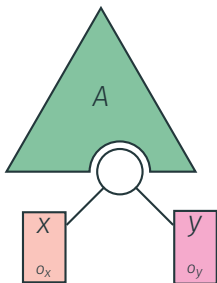
Il existe un arbre optimum dans lequel les deux plus petites occurrences sont « frères ».

Propriétés des arbres de compression optimaux

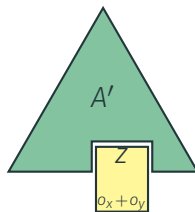
Lemme

Soit A un arbre dans lequel les deux plus petites occurrences x et y sont « frères ». Soit A' l'arbre où le père de x et y est remplacé par un symbole $x + y$ poids somme.

Alors A est un arbre de compression optimal ssi A' est un arbre de compression optimal



devient



$$L(A) = L(A') + o_x + o_y$$

Algorithme de Huffman

~1952



David Huffman
(1925-1999)

Algorithme 1 : Algorithme de Huffman

Entrées : occurrences des symboles $(o_\sigma)_{\sigma \in \Sigma}$

Sorties : Arbre de compression optimal

- 1 $F \leftarrow$ file à priorité vide
 - 2 **pour** $\sigma \in \Sigma$ **faire**
 - 3 | Enfiler un arbre feuille d'étiquette (σ, o_σ) .
 - 4 **tant que** F *contient* ≥ 2 *arbres* **faire**
 - 5 | Extraire les deux arbres A_1 et A_2 de plus
 | petit poids o_1 et o_2 .
 - 6 | Enfiler l'arbre $\text{Nœud}(A_1, A_2)$ de poids $o_1 + o_2$.
 - 7 **retourner** *Sommet de la file*
-

Algorithme de Huffman : illustration

Exemple

Considérons $\Sigma = \{a, b, c, d, e, f, g, h\}$ avec les occurrences

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
occurrences	1098	111	536	602	3030	146	204	125

Algorithme de Huffman : illustration

Exemple

a	b	c	d	e	f	g	h
1098	111	536	602	3030	146	204	125

Algorithme de Huffman : illustration

Exemple

e	a	d	c	g	f	h	b
3030	1098	602	536	204	146	125	111

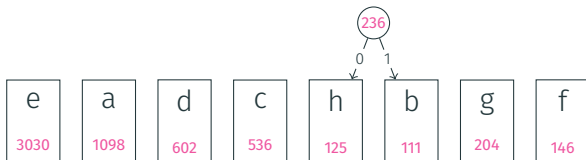
Algorithme de Huffman : illustration

Exemple



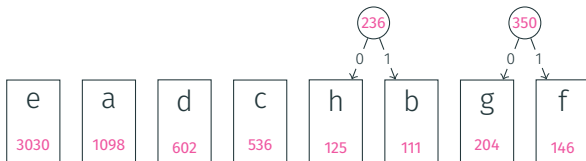
Algorithme de Huffman : illustration

Exemple



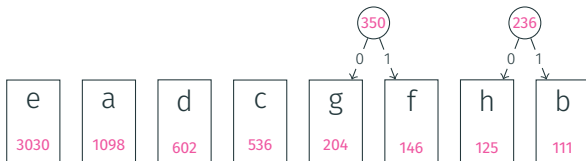
Algorithme de Huffman : illustration

Exemple



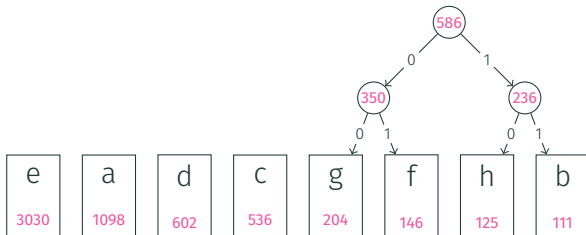
Algorithme de Huffman : illustration

Exemple



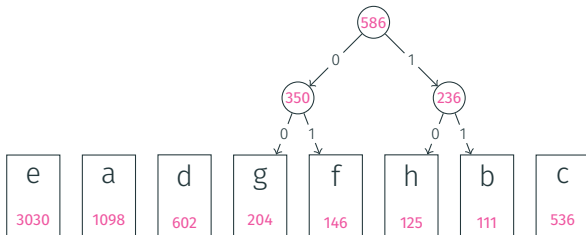
Algorithme de Huffman : illustration

Exemple



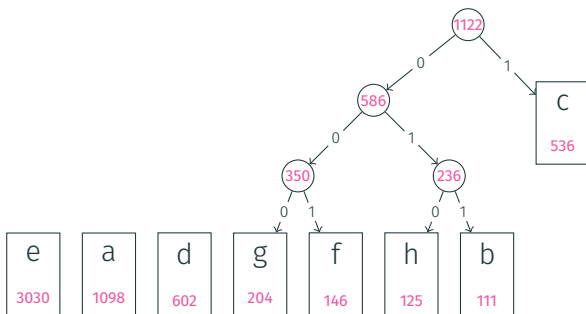
Algorithme de Huffman : illustration

Exemple



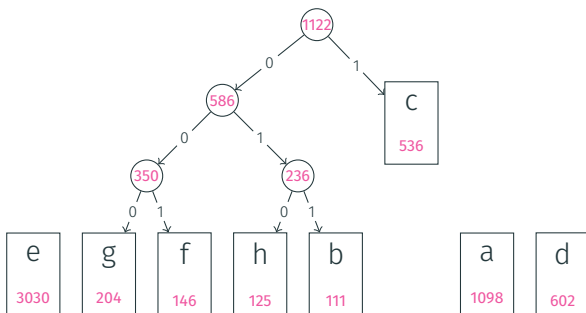
Algorithme de Huffman : illustration

Exemple



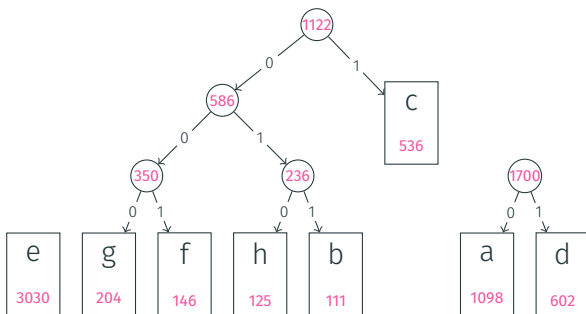
Algorithme de Huffman : illustration

Exemple



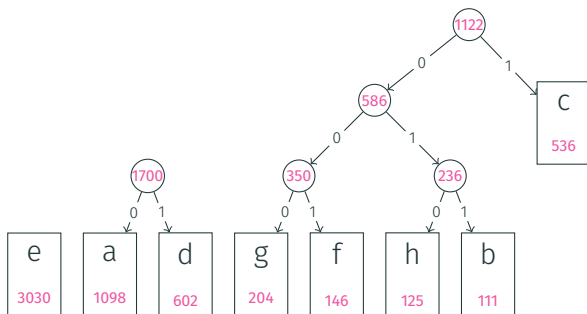
Algorithme de Huffman : illustration

Exemple



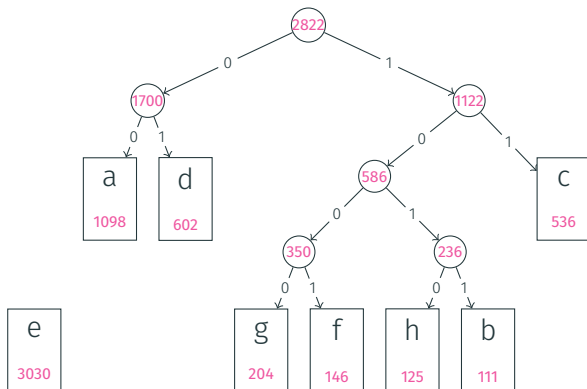
Algorithme de Huffman : illustration

Exemple



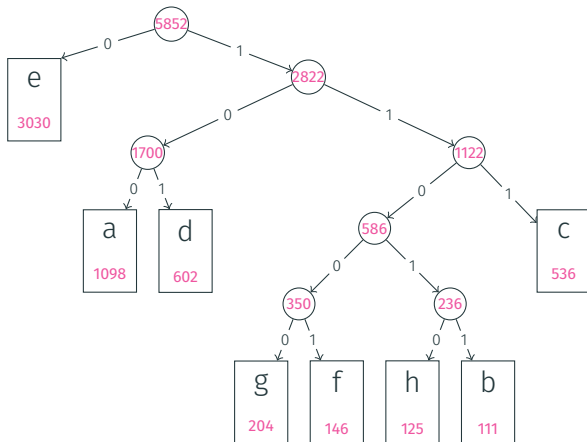
Algorithme de Huffman : illustration

Exemple



Algorithme de Huffman : illustration

Exemple



Algorithme de Huffman : illustration

Exemple

On obtient ainsi le code optimum

σ	$E(\sigma)$	σ	$E(\sigma)$
a	100	e	0
b	11011	f	11001
c	111	g	11000
d	101	h	11010

La longueur du texte est

$$(204 + 146 + 125 + 111) \cdot 5 + (1098 + 602 + 536) \cdot 3 + 3030 \cdot 1 = 12668.$$